

Elementary Sort!

Selection sort demo:

- In iteration 1, find index min of smallest remaining entry
- Swap $a[i]$ and $a[\text{min}]$



Initial

Selection sort demo:

- In iteration i , find index $\min$ of smallest remaining entry
- Swap $a[i]$ and $a[\min]$

i



Remaining
entries

Selection sort demo:

- In iteration i , find index $\min$ of smallest remaining entry
- Swap $a[i]$ and $a[\min]$

i

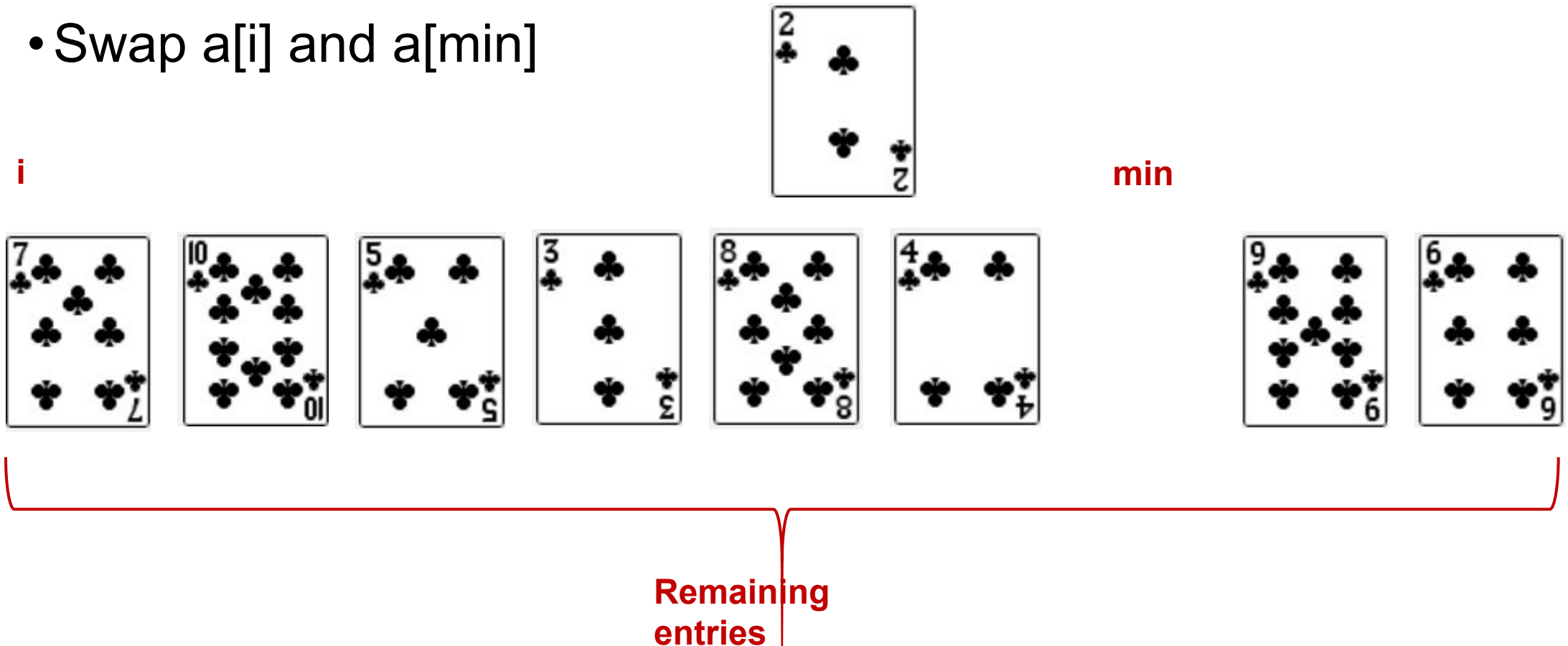
$\min$



Remaining
entries

Selection sort demo:

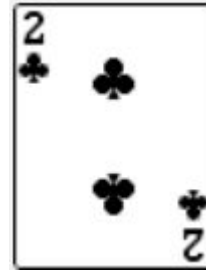
- In iteration 1, find index min of smallest remaining entry
- Swap $a[i]$ and $a[\text{min}]$



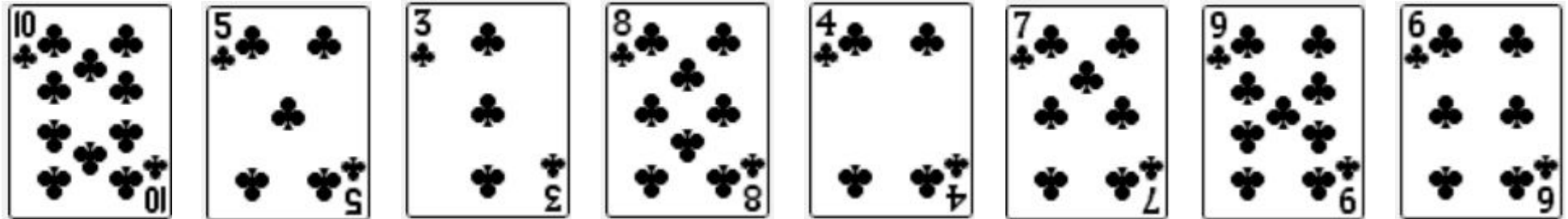
Selection sort demo:

- In iteration 1, find index min of smallest remaining entry
- Swap $a[i]$ and $a[\text{min}]$

i



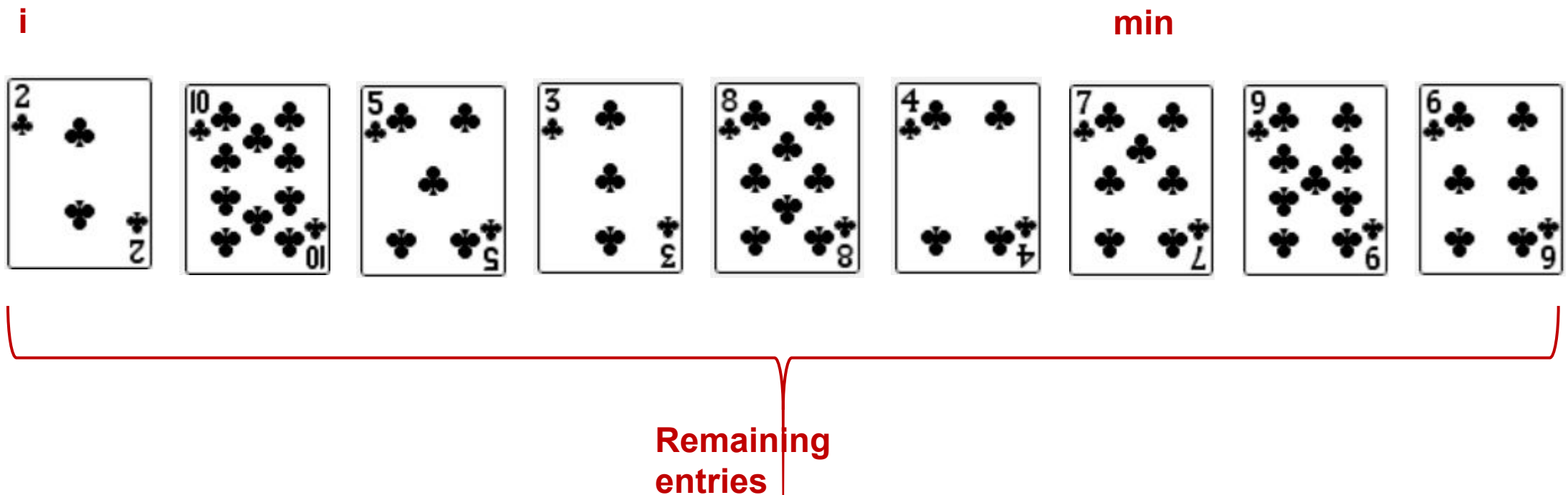
min



**Remaining
entries**

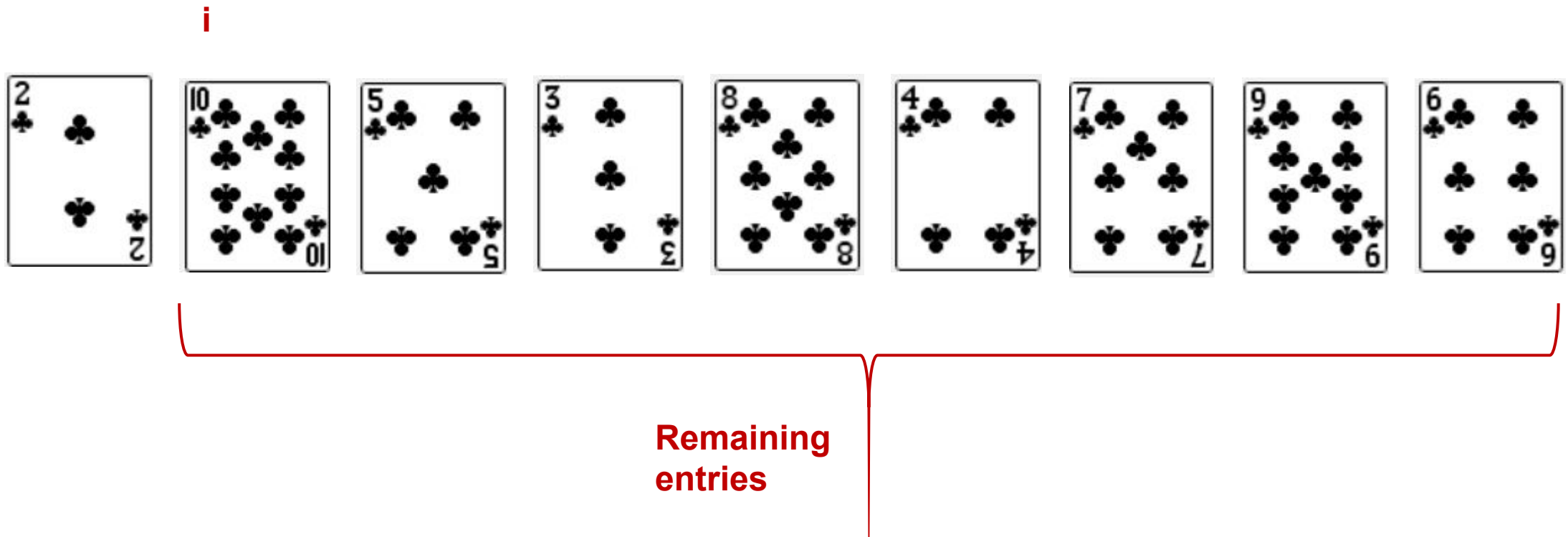
Selection sort demo:

- In iteration 1, find index min of smallest remaining entry
- Swap $a[i]$ and $a[\text{min}]$



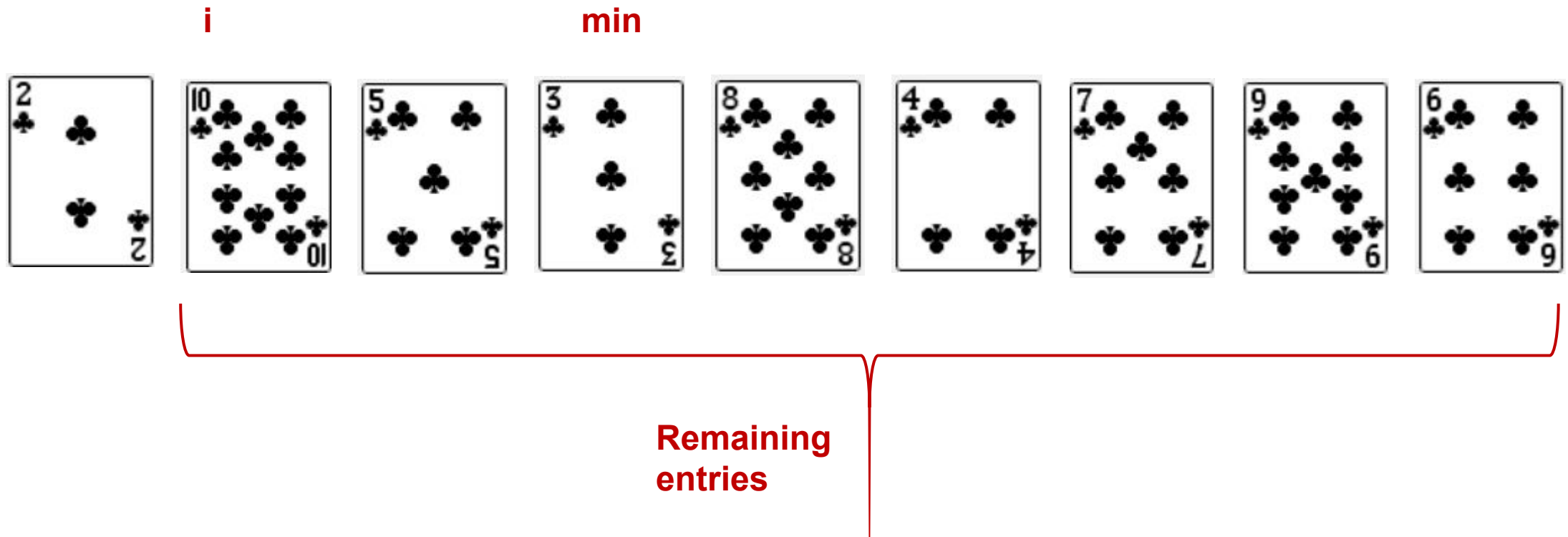
Selection sort demo:

- In iteration i , find index $\min$ of smallest remaining entry
- Swap $a[i]$ and $a[\min]$



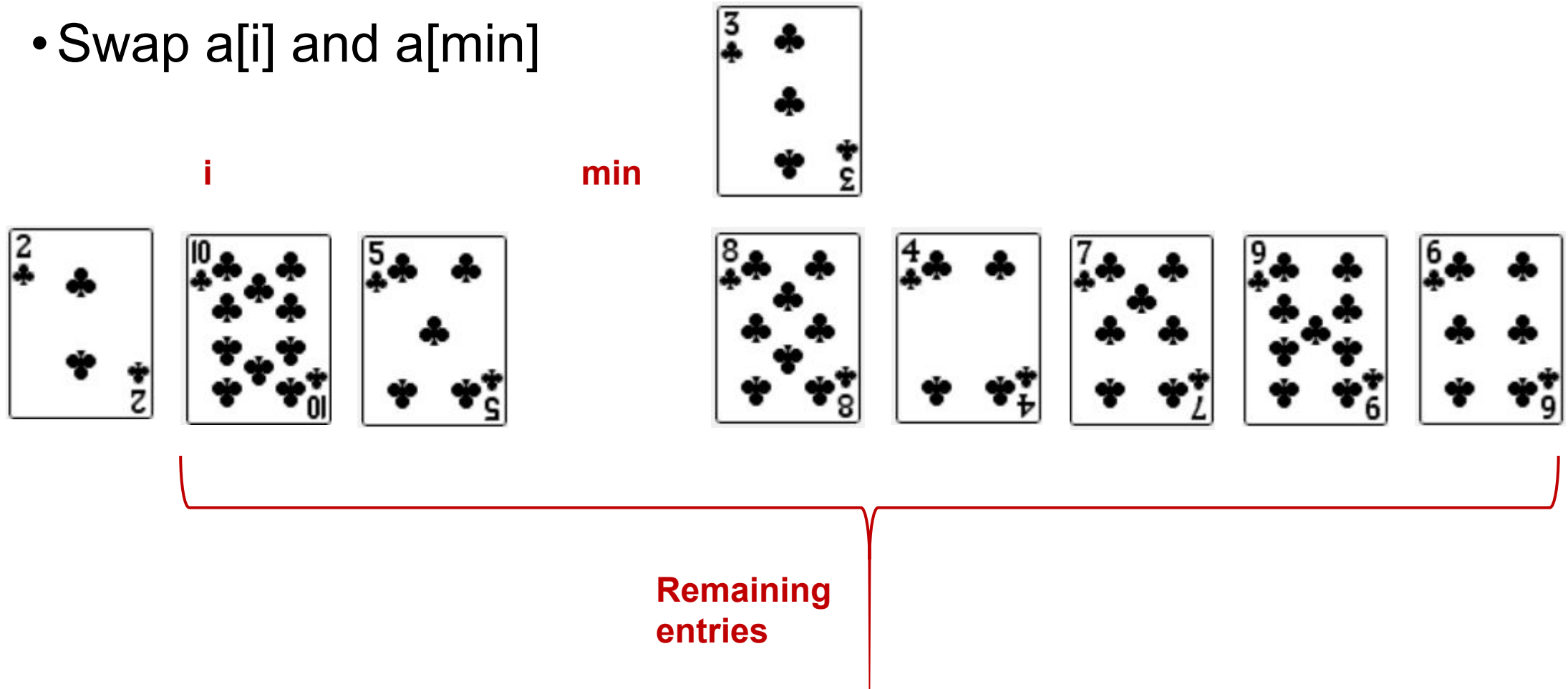
Selection sort demo:

- In iteration i , find index min of smallest remaining entry
- Swap $a[i]$ and $a[\text{min}]$



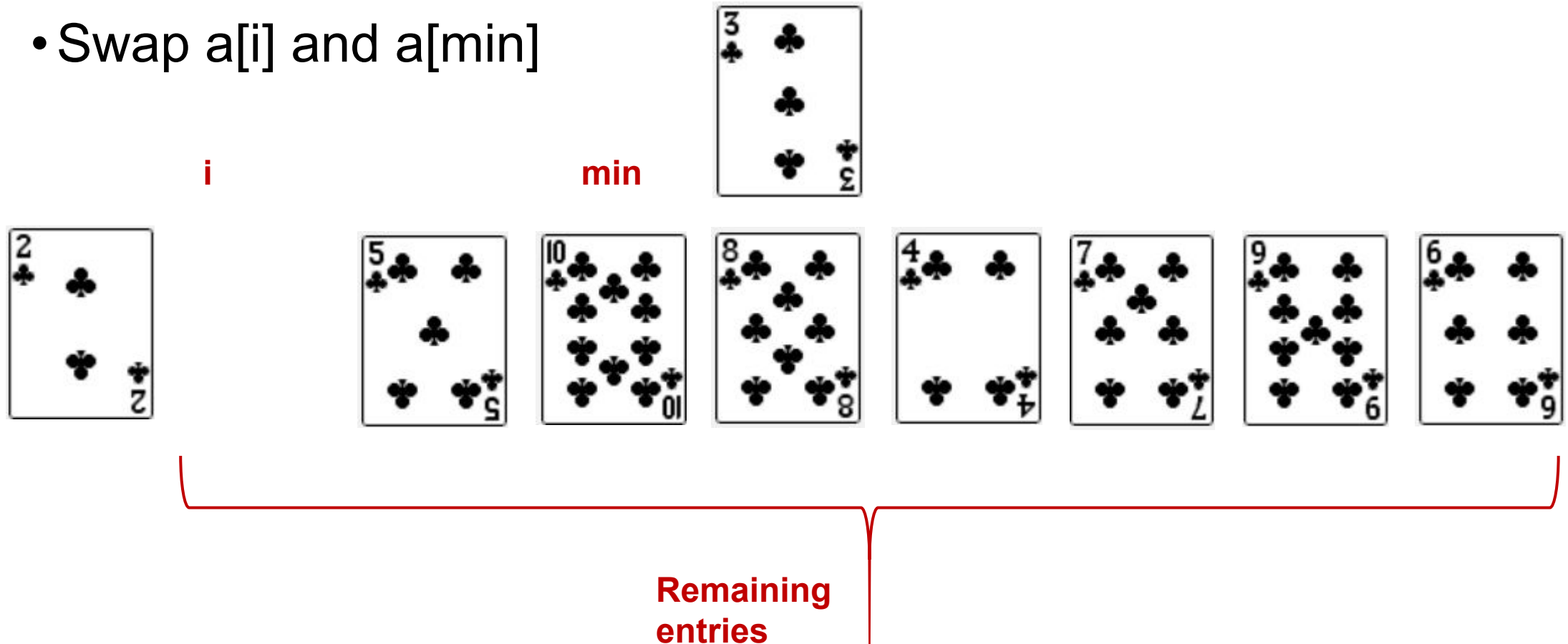
Selection sort demo:

- In iteration 1, find index min of smallest remaining entry
- Swap $a[i]$ and $a[\text{min}]$



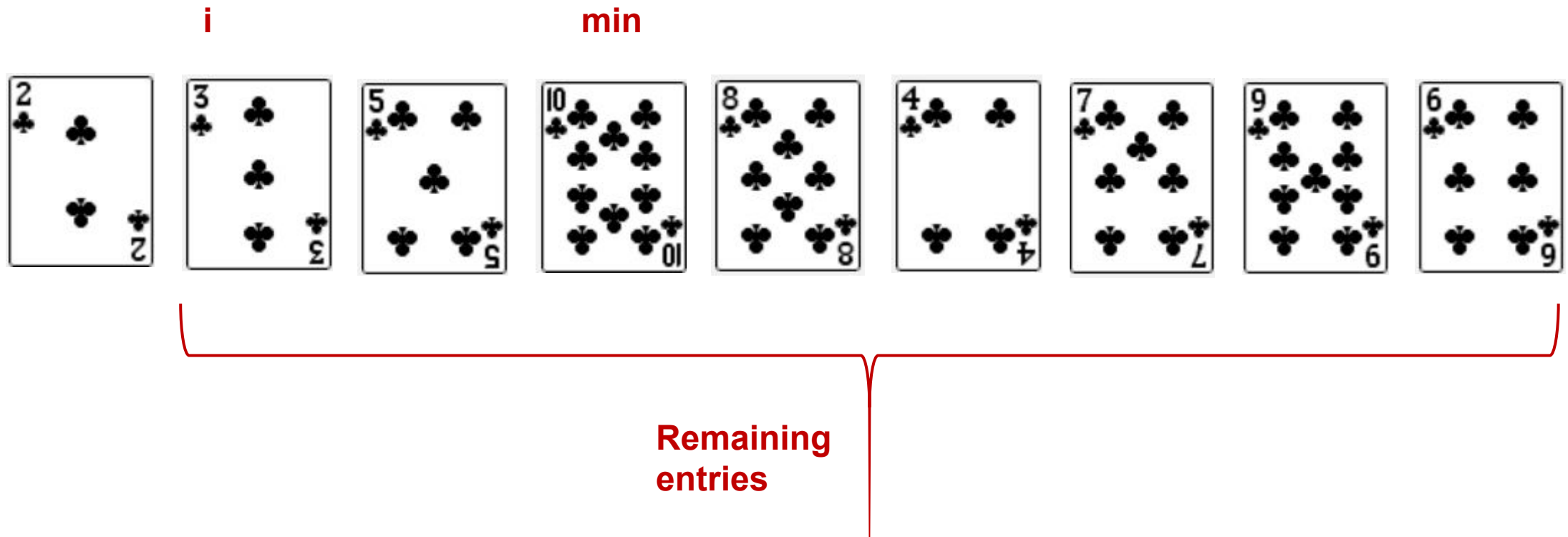
Selection sort demo:

- In iteration i , find index min of smallest remaining entry
- Swap $a[i]$ and $a[\text{min}]$



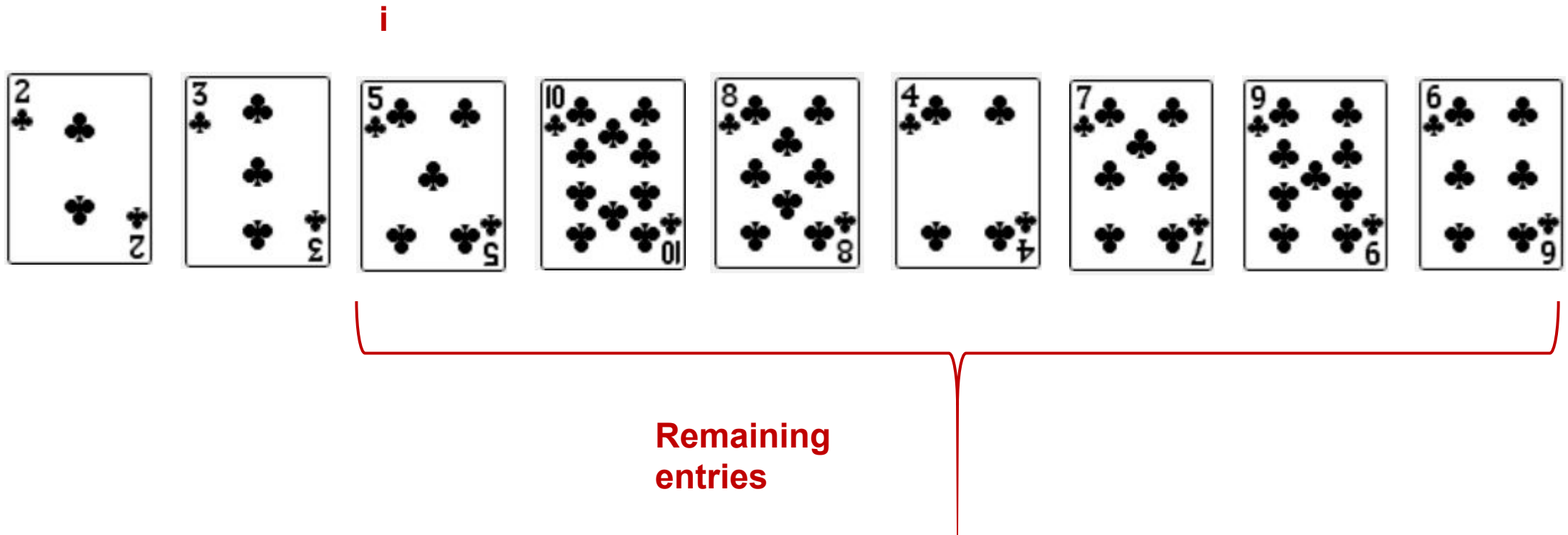
Selection sort demo:

- In iteration 1, find index min of smallest remaining entry
- Swap $a[i]$ and $a[\text{min}]$



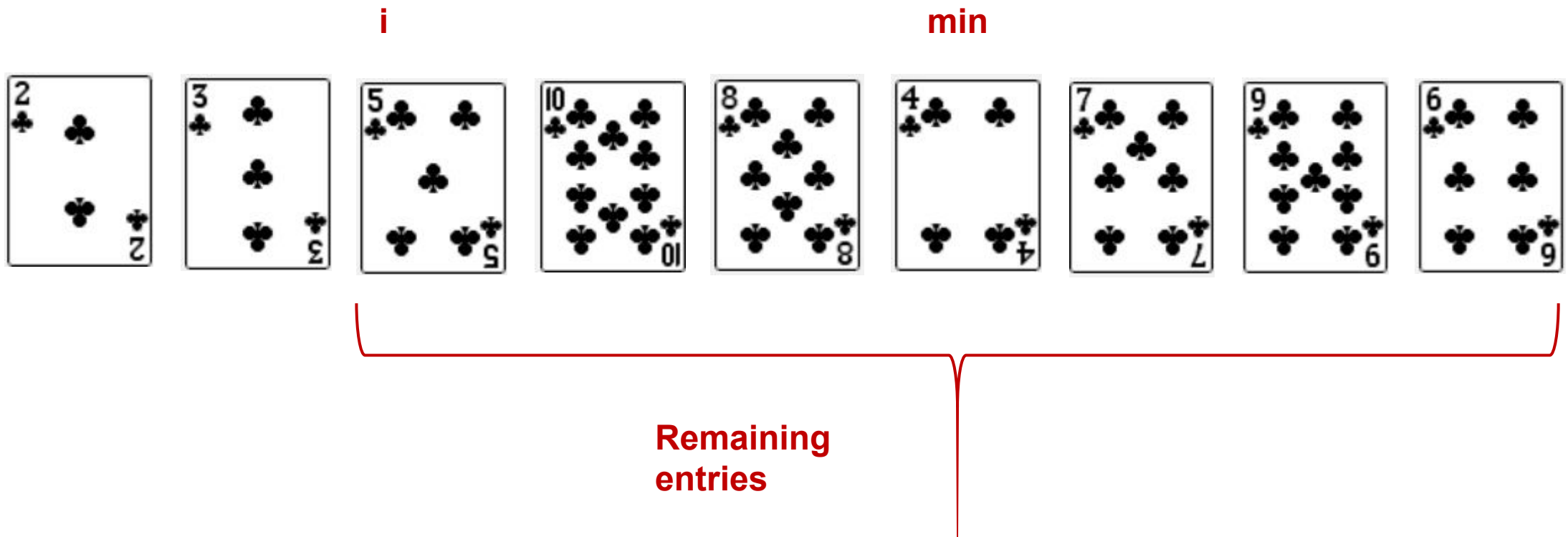
Selection sort demo:

- In iteration i , find index $\min$ of smallest remaining entry
- Swap $a[i]$ and $a[\min]$



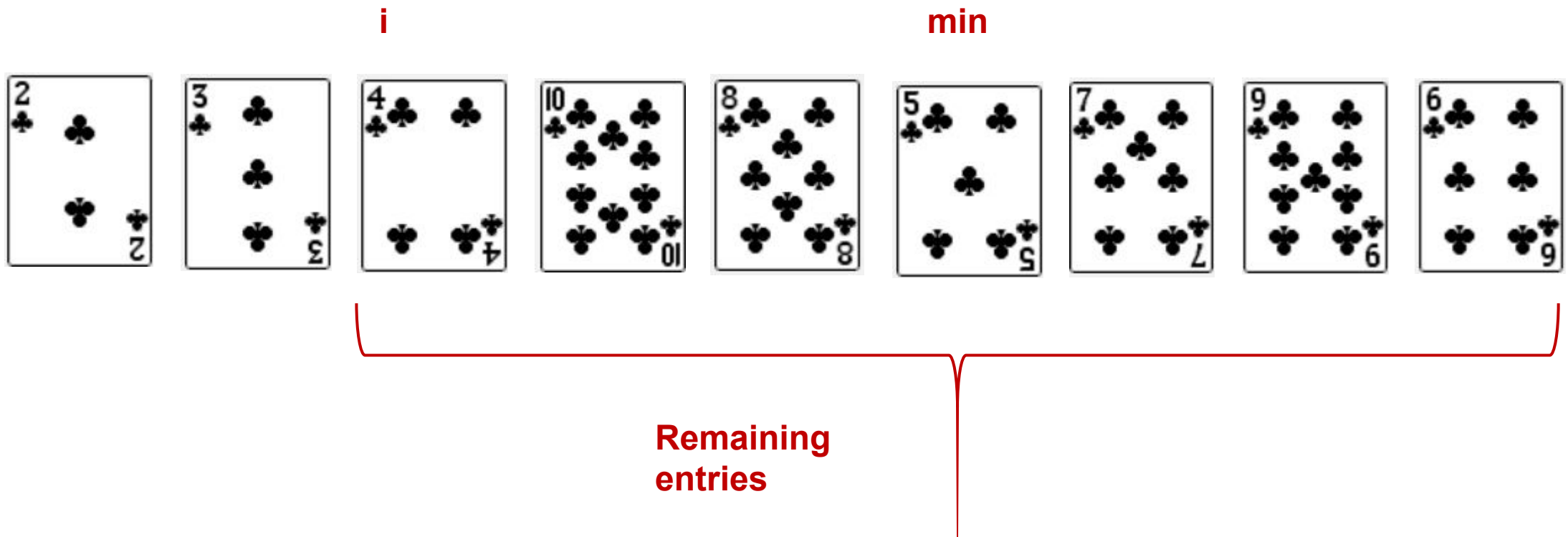
Selection sort demo:

- In iteration 1, find index min of smallest remaining entry
- Swap $a[i]$ and $a[\text{min}]$



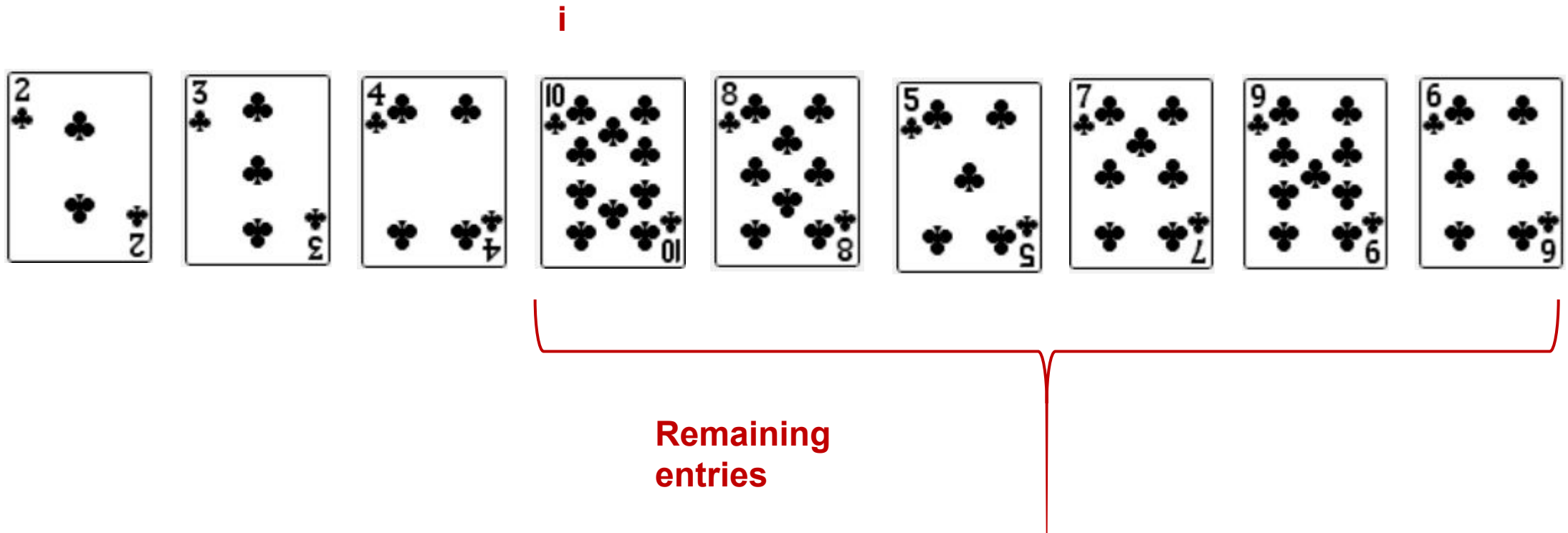
Selection sort demo:

- In iteration i , find index $\min$ of smallest remaining entry
- Swap $a[i]$ and $a[\min]$



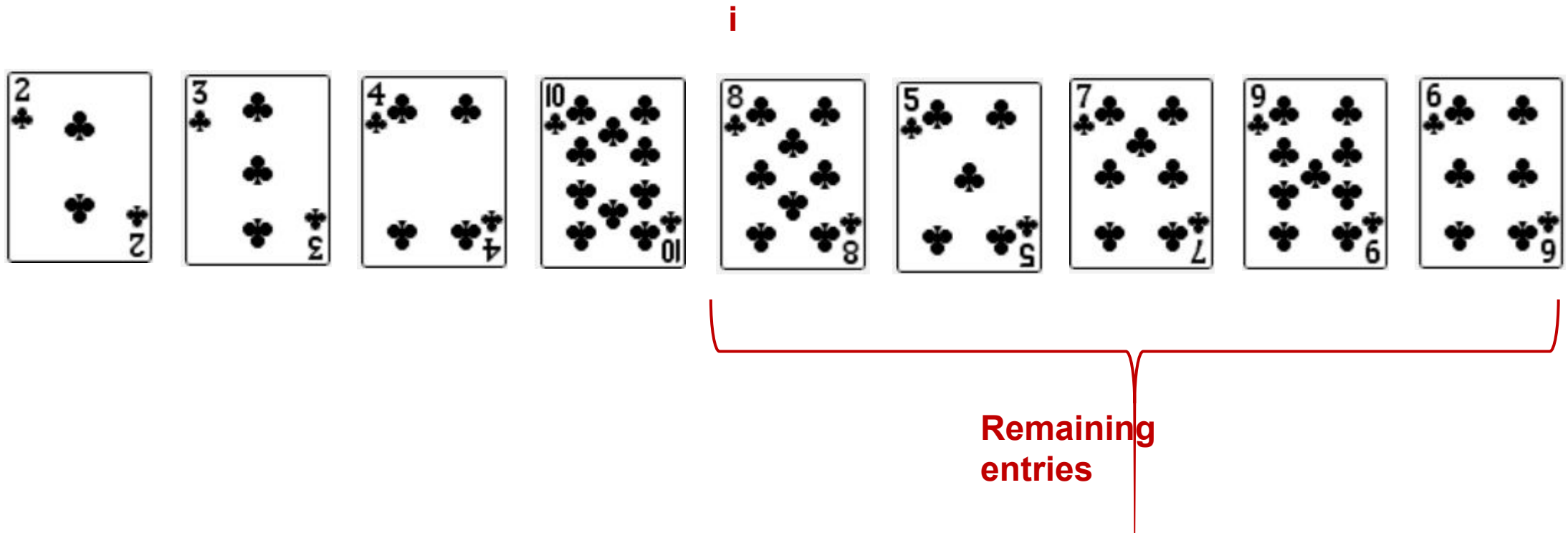
Selection sort demo:

- In iteration i , find index $\min$ of smallest remaining entry
- Swap $a[i]$ and $a[\min]$



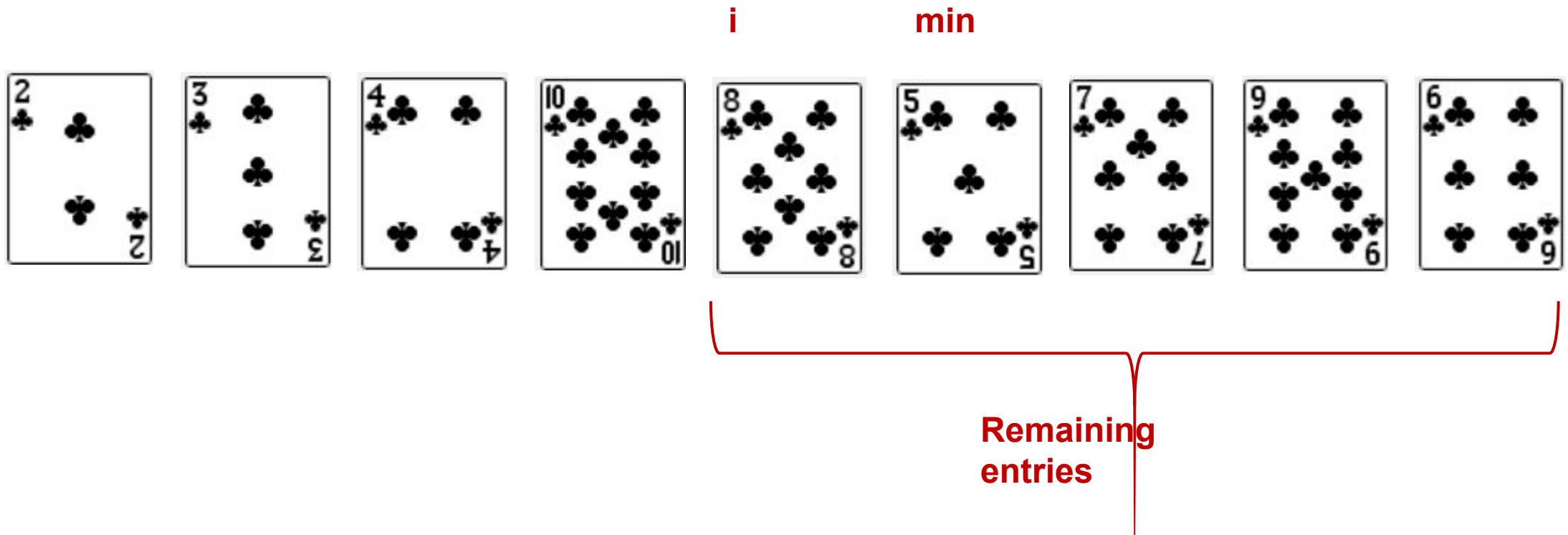
Selection sort demo:

- In iteration i , find index min of smallest remaining entry
- Swap $a[i]$ and $a[\text{min}]$



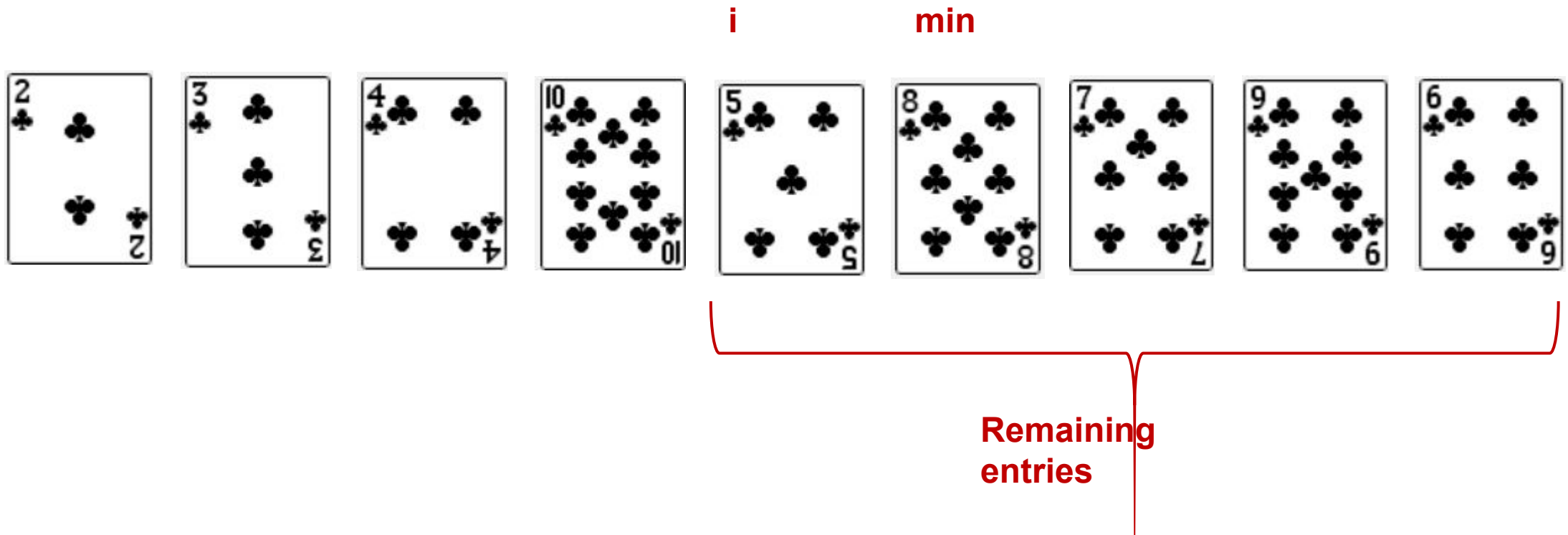
Selection sort demo:

- In iteration i , find index min of smallest remaining entry
- Swap $a[i]$ and $a[\text{min}]$



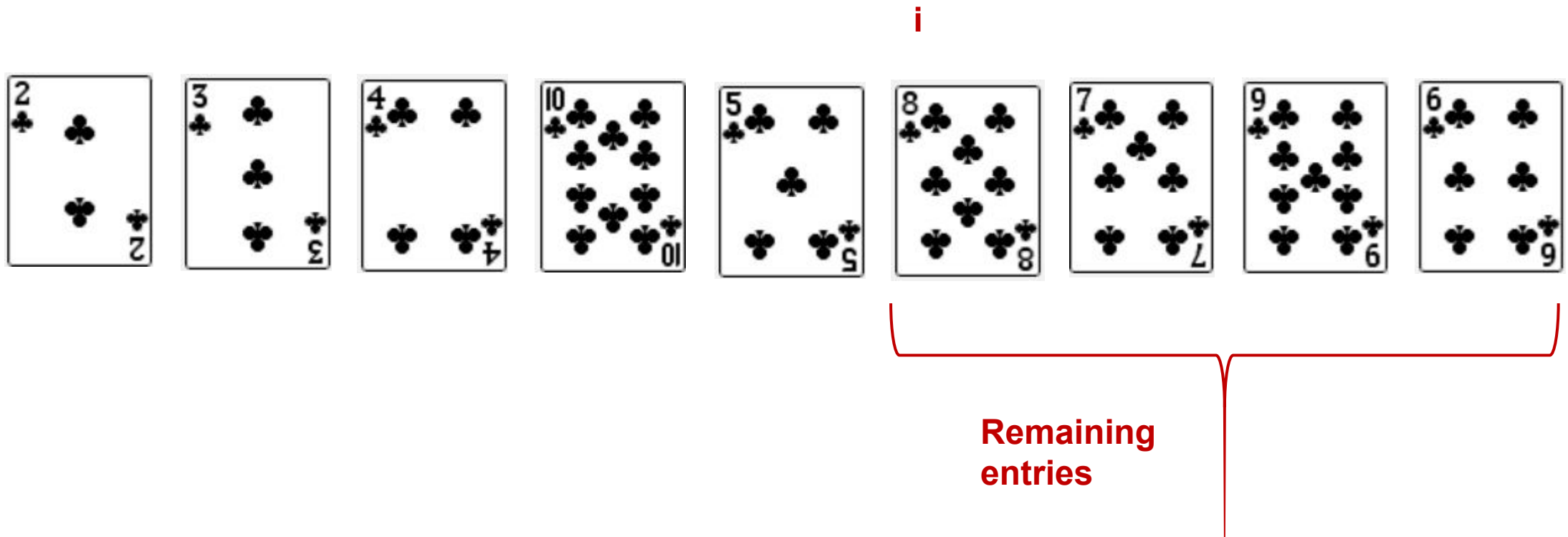
Selection sort demo:

- In iteration i , find index min of smallest remaining entry
- Swap $a[i]$ and $a[\text{min}]$



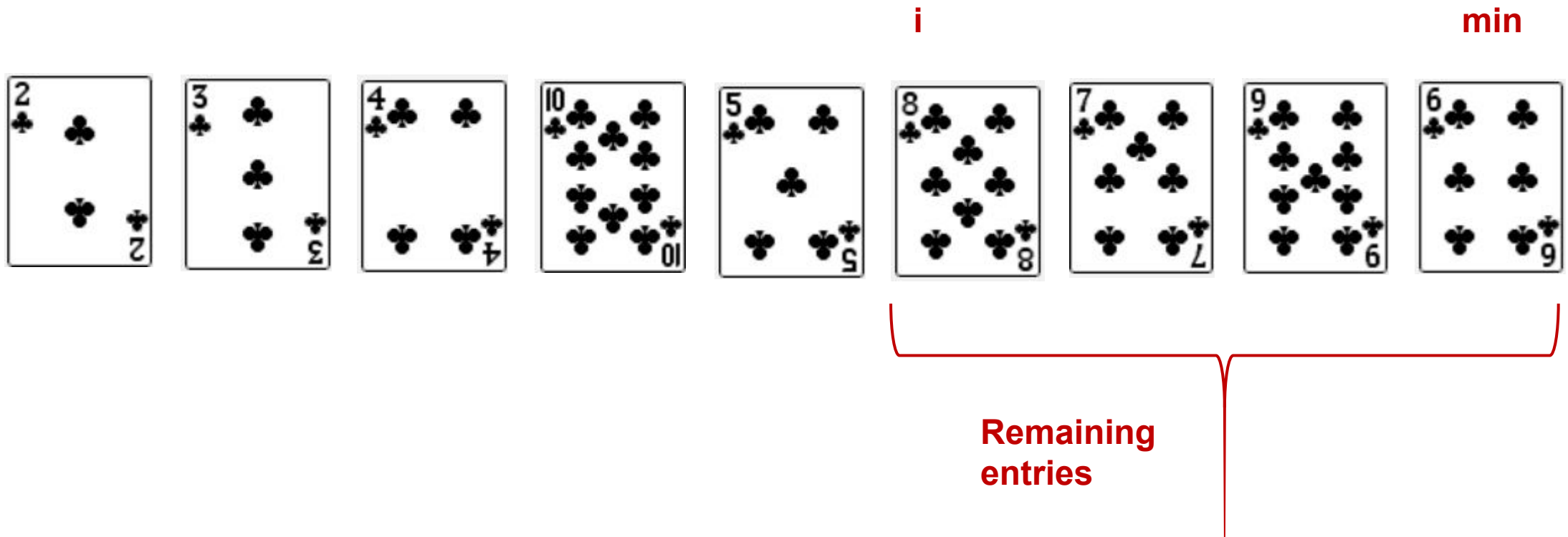
Selection sort demo:

- In iteration i , find index $\min$ of smallest remaining entry
- Swap $a[i]$ and $a[\min]$



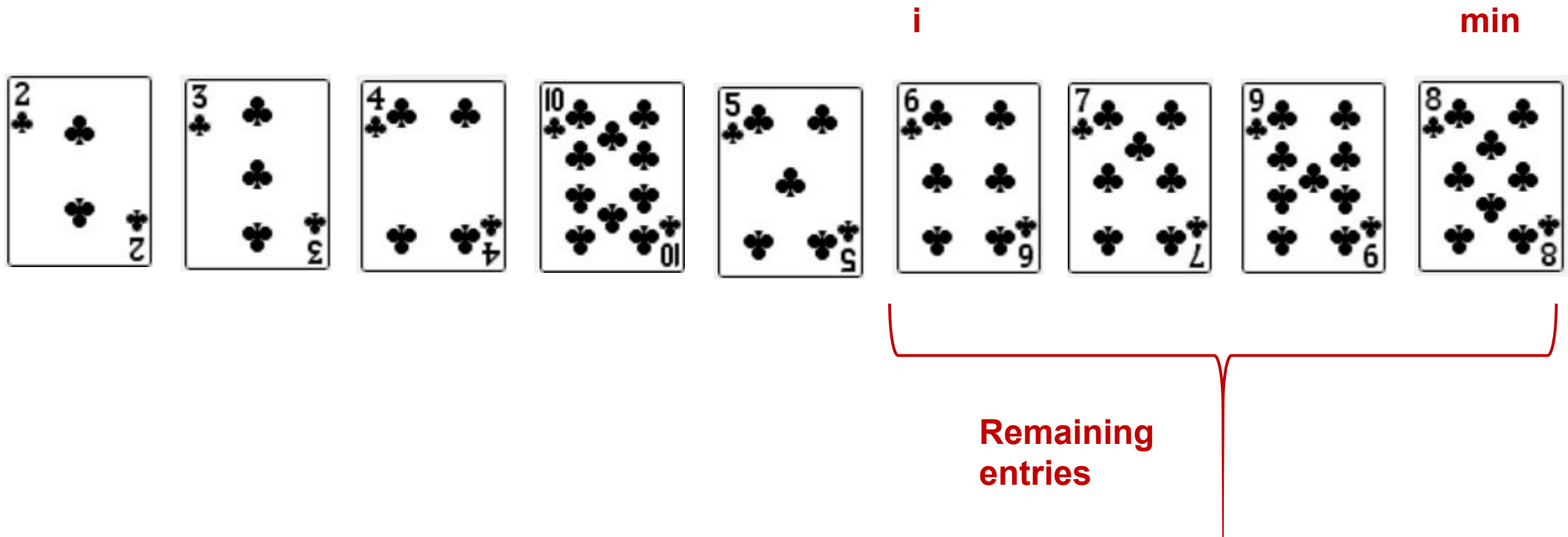
Selection sort demo:

- In iteration i , find index min of smallest remaining entry
- Swap $a[i]$ and $a[\text{min}]$



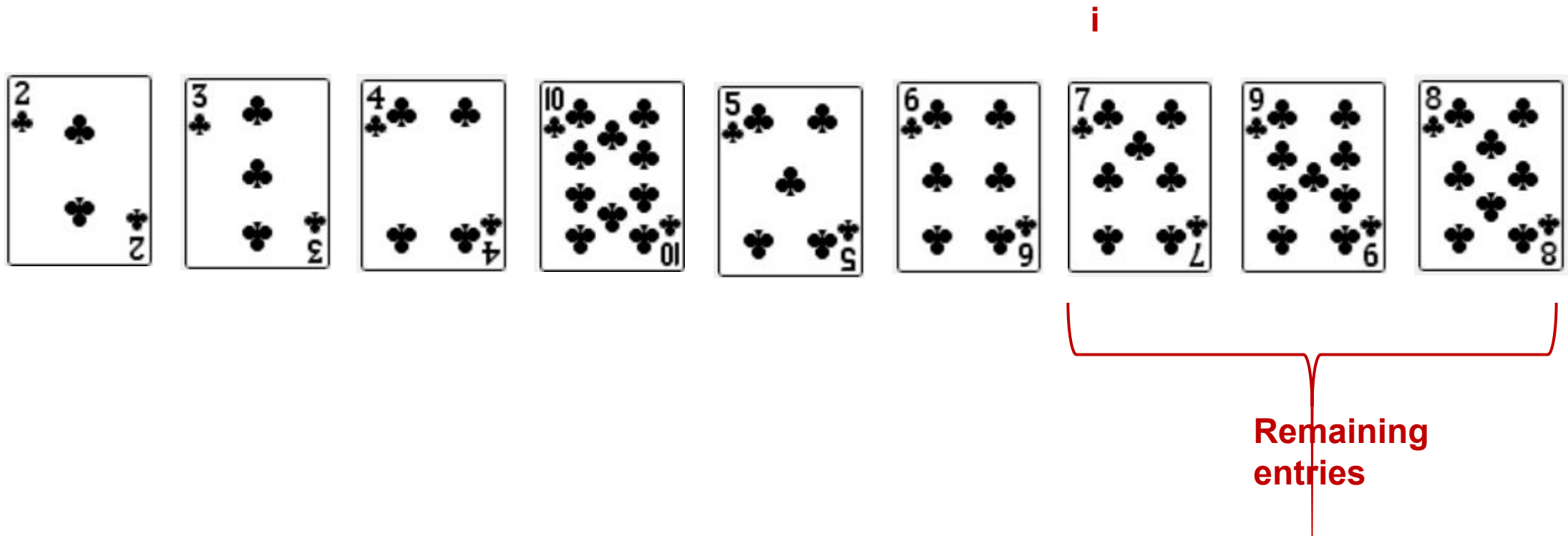
Selection sort demo:

- In iteration i , find index min of smallest remaining entry
- Swap $a[i]$ and $a[\text{min}]$



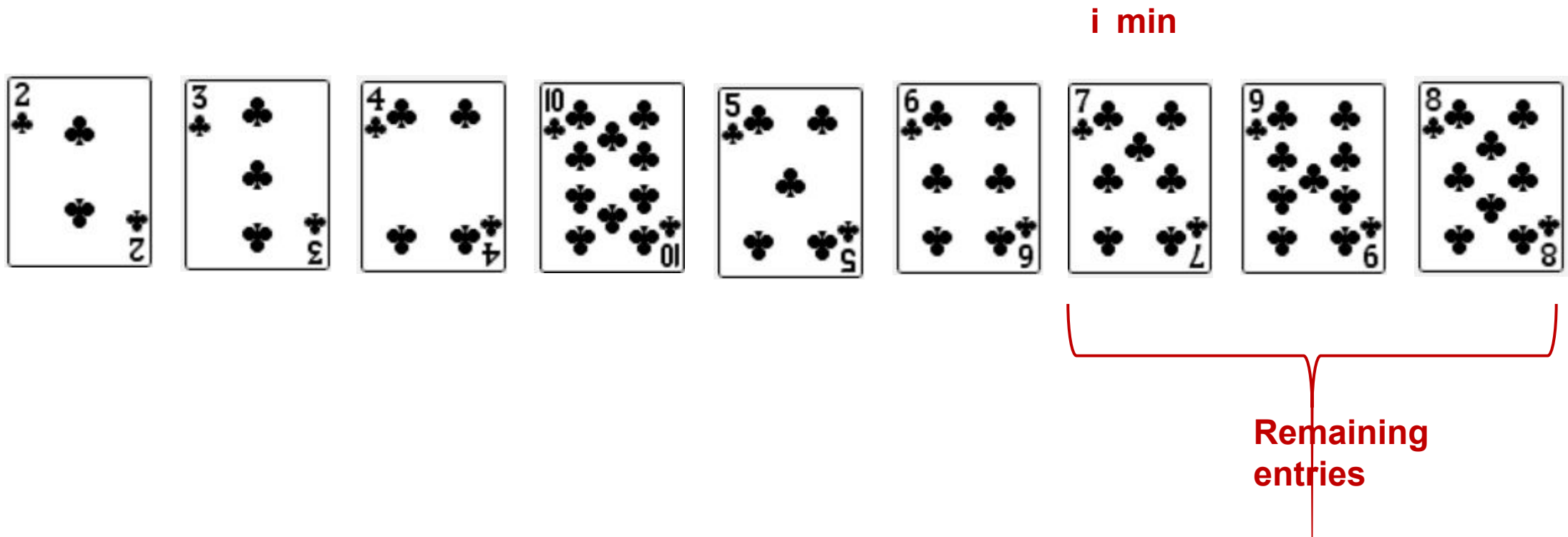
Selection sort demo:

- In iteration i , find index $\min$ of smallest remaining entry
- Swap $a[i]$ and $a[\min]$



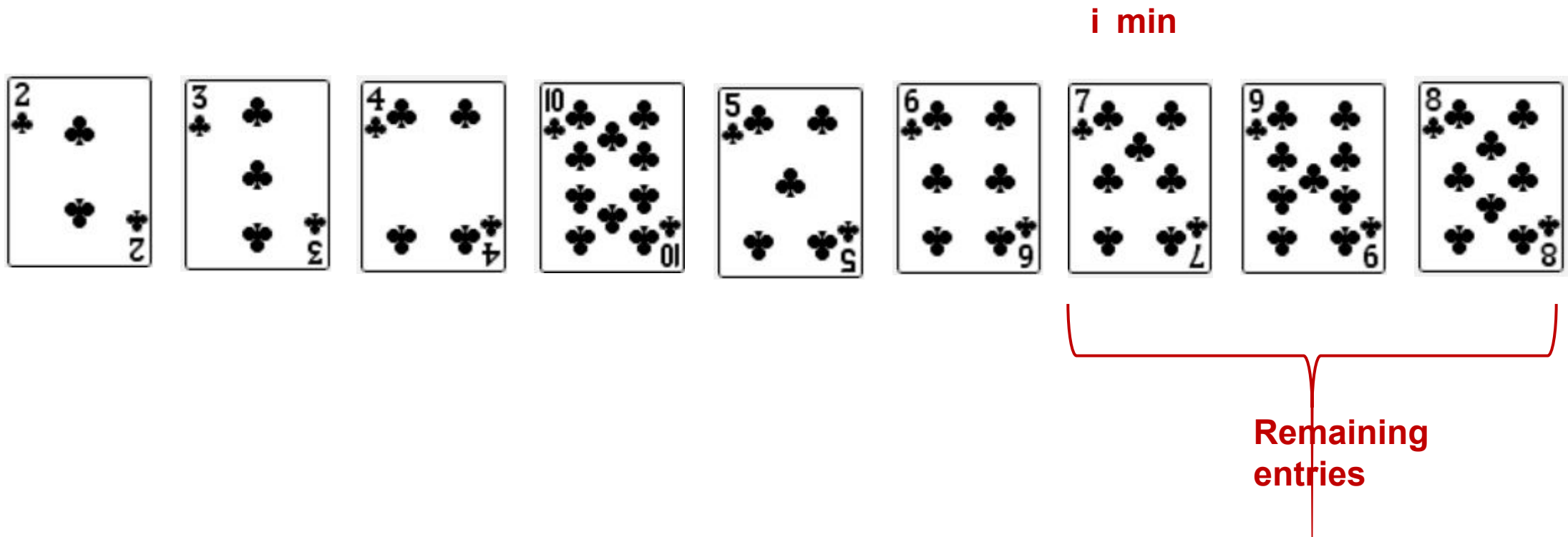
Selection sort demo:

- In iteration i , find index $\min$ of smallest remaining entry
- Swap $a[i]$ and $a[\min]$



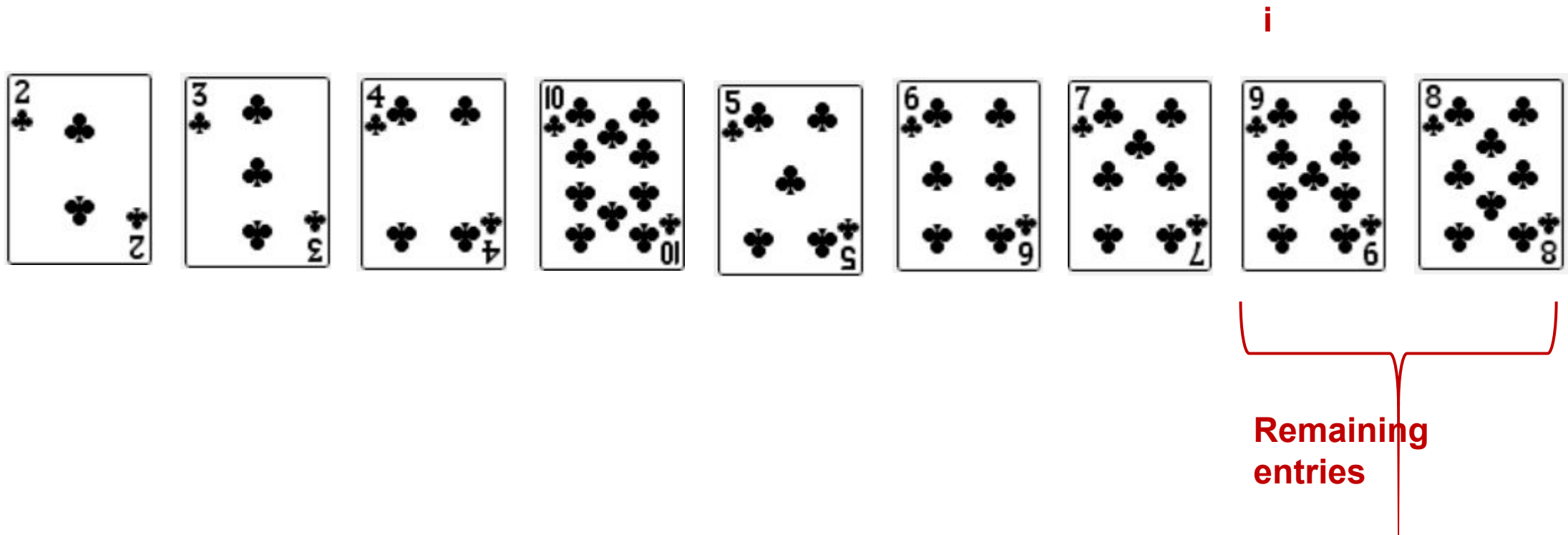
Selection sort demo:

- In iteration i , find index $\min$ of smallest remaining entry
- Swap $a[i]$ and $a[\min]$



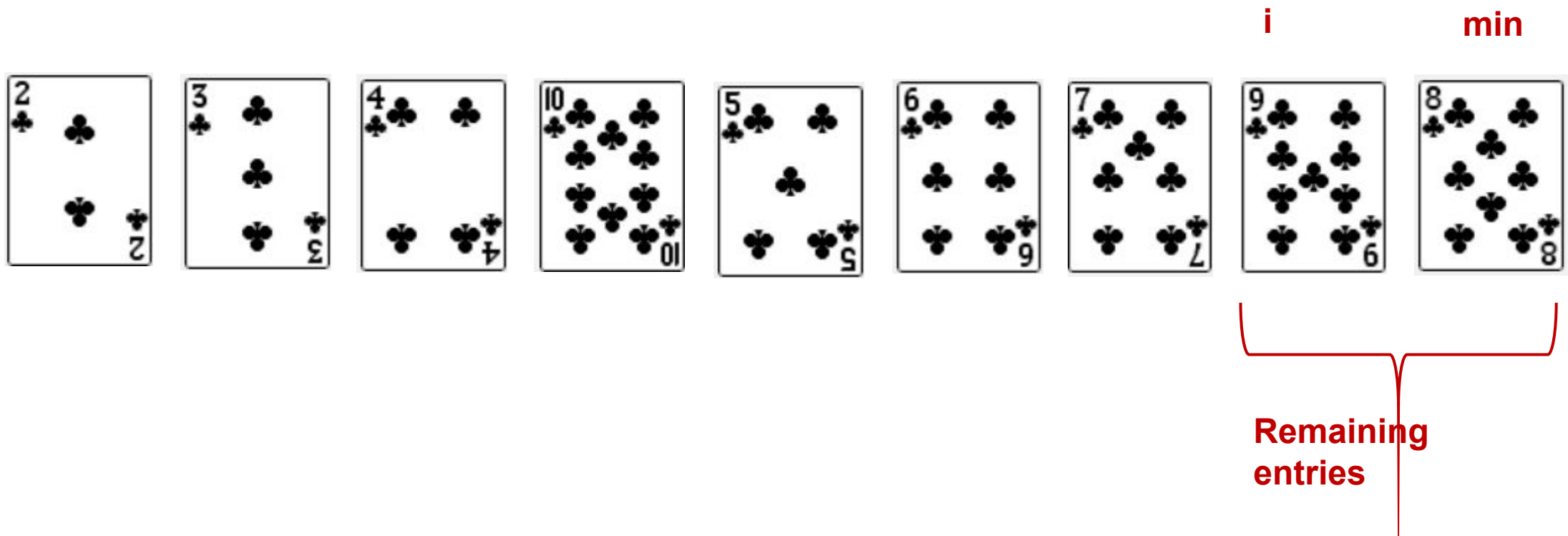
Selection sort demo:

- In iteration i , find index $\min$ of smallest remaining entry
- Swap $a[i]$ and $a[\min]$



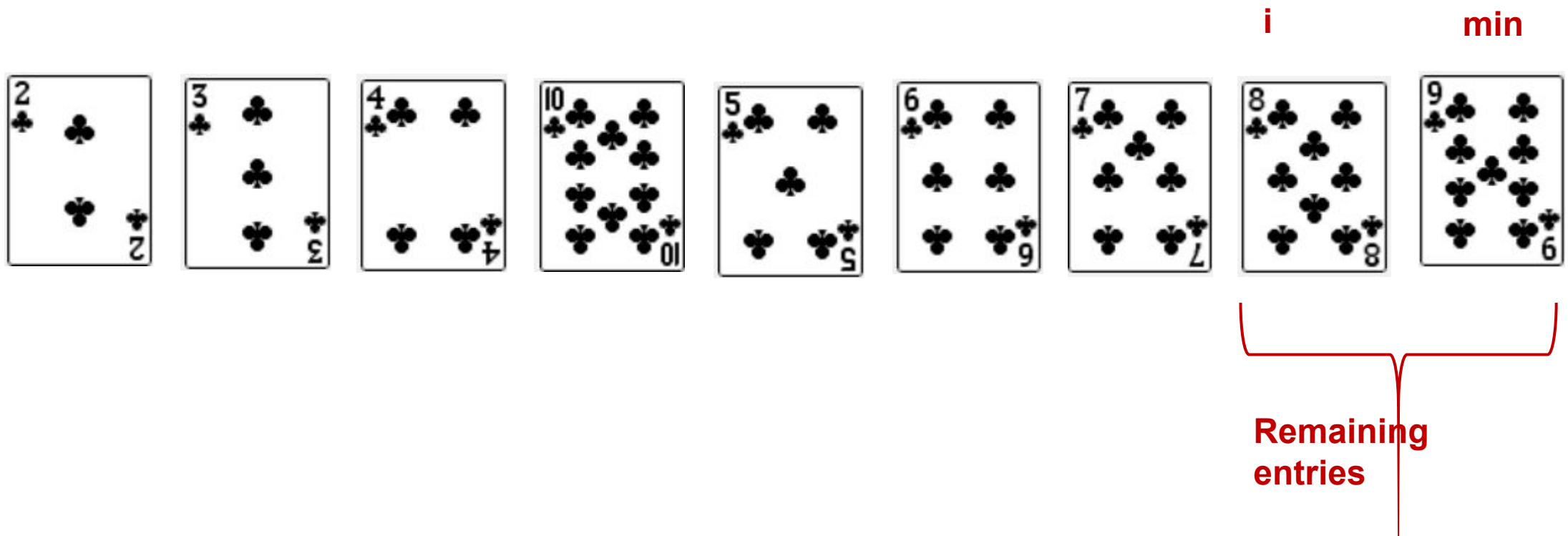
Selection sort demo:

- In iteration i , find index min of smallest remaining entry
- Swap $a[i]$ and $a[\text{min}]$



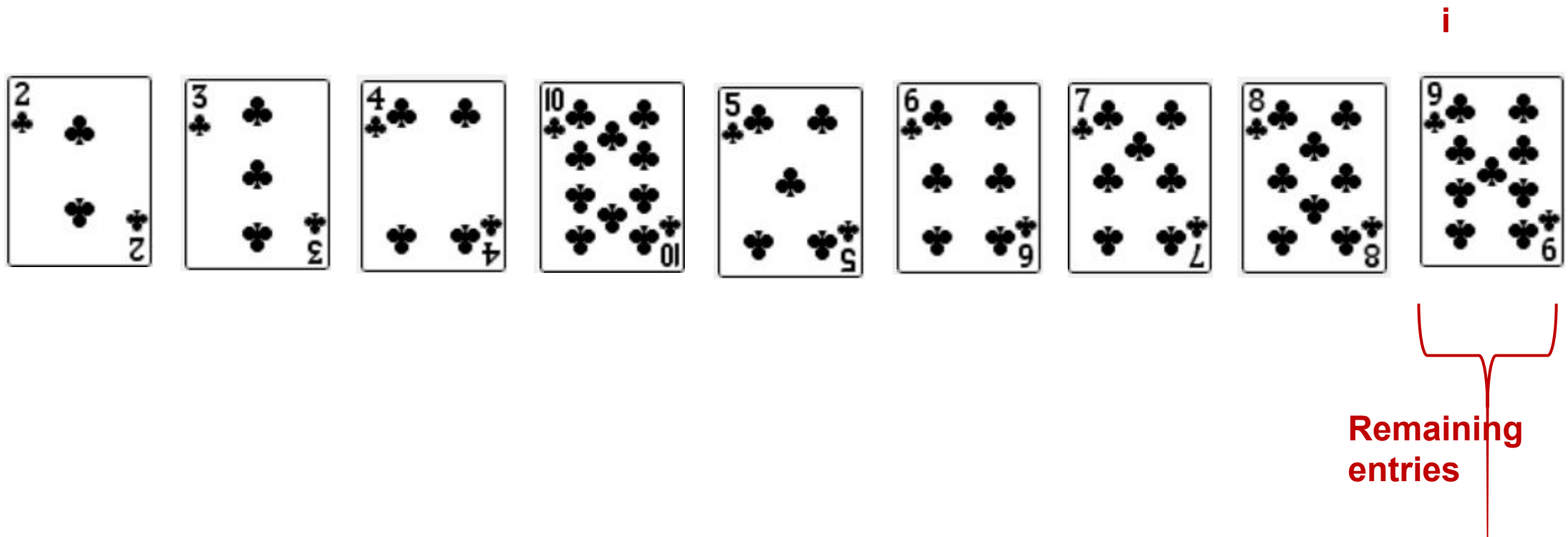
Selection sort demo:

- In iteration i , find index min of smallest remaining entry
- Swap $a[i]$ and $a[\text{min}]$



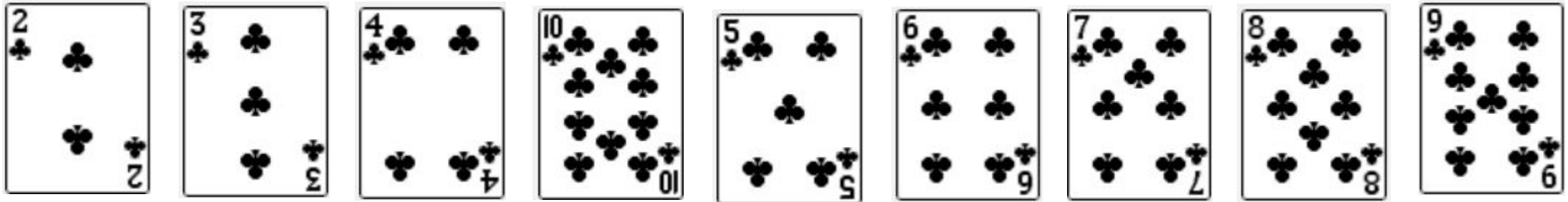
Selection sort demo:

- In iteration i , find index $\min$ of smallest remaining entry
- Swap $a[i]$ and $a[\min]$



Selection sort demo:

- In iteration i , find index $\min$ of smallest remaining entry
- Swap $a[i]$ and $a[\min]$



**final
order**

Selection sort:

Algorithm: ↑ scan from left to right

Invariants:

- Entries that left of ↑ (including ↑) fixed and in ascending order.
- No entry to right of ↑ is smaller than any entry to the left of ↑



Selection sort inner loop

To maintain algorithm invariants:

- Move the pointer to the right

$i += 1$



Selection sort inner loop

To maintain algorithm invariants:

- Move the pointer to the right

$i += 1$

- Identify the index of minimum entry or right

$\text{min} = i$

for j range($i+1, n$):

if $a[j] < a[\text{min}]$: $\text{min} = j$



Selection sort inner loop

To maintain algorithm invariants:

- Move the pointer to the right
 $i += 1$
- Identify the index of minimum entry or right
 $\text{min} = i$
for j range($i+1, n$):
 if $a[j] < a[\text{min}]$: $\text{min} = j$
- Exchange:
 $a[i], a[\text{min}] = a[\text{min}], a[i]$



Selection sort: Python implementation

```
def selectionSort(a):  
    n = size(a)  
    for i in range(n):  
        min = i  
        for j in range(i+1, n):  
            if a[j] < a[min]:  
                min = j  
        a[i], a[min] = a[min], a[i]
```

Selection sort: mathematical analysis

Proposition. Selection sort uses $(N-1)+(N-2)+\dots+1+0 \sim N^2/2$ compares and N exchanges

i	min	0	1	2	3	4	5	6	7	8	9	10
		S	O	R	T	E	X	A	M	P	L	E
0	6	A	O	R	T	E	X	S	M	P	L	E
1	4	A	E	R	T	O	X	S	M	P	L	E
2	10	A	E	E	T	O	X	S	M	P	L	R
3	9	A	E	E	L	O	X	S	M	P	T	R
4	7	A	E	E	L	M	X	S	O	P	T	R
5	7	A	E	E	L	M	O	S	X	P	T	R
6	8	A	E	E	L	M	O	P	X	S	T	R
7	10	A	E	E	L	M	O	P	S	X	T	R
8	8	A	E	E	L	M	O	P	R	S	T	X
9	9	A	E	E	L	M	O	P	R	S	T	X
10	10	A	E	E	L	M	O	P	R	S	T	X

entries in black are examined to find the minimum

entries in red are a[minimum]

entries in gray are in final position

A E E L M O P R S T X

Trace of selection sort (array contents just after each exchange)

QUIZ

How many compares does selection sort make when the input array is *already sorted*?

- a) Logarithmic
- b) Linear
- c) Quadratic
- d) exponential

QUIZ

How many compares does selection sort make when the input array is *already sorted*?

- a) Logarithmic
- b) Linear
- c) Quadratic
- d) Exponential

The number of compares and exchanges made by selection sort does not depend on the order of the input. Ans - Quadratic

Insertion sort

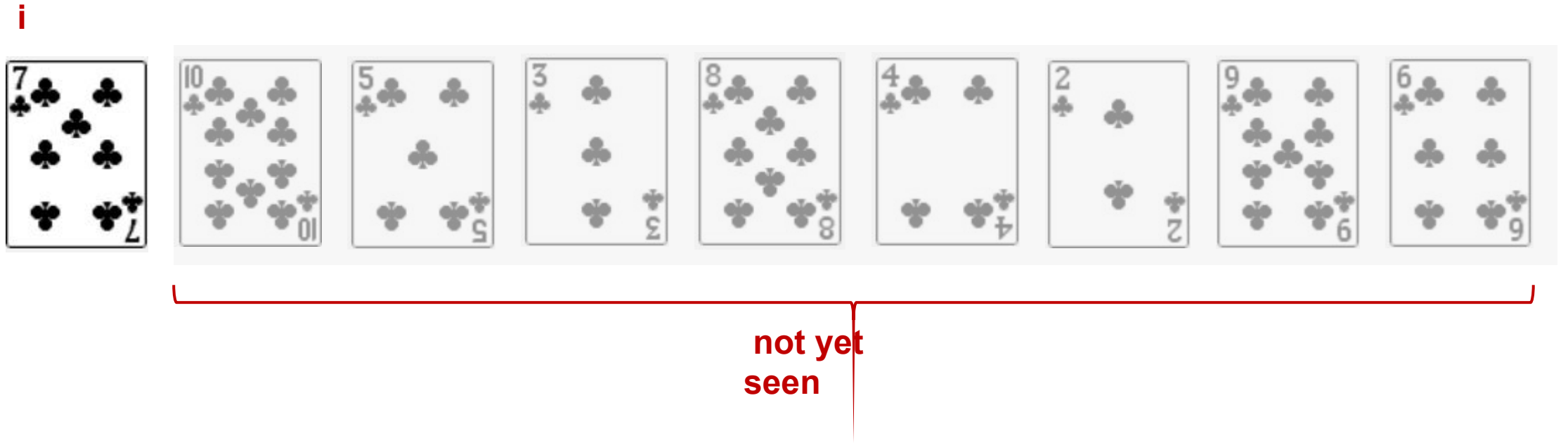
- In iteration i , swap $a[i]$ with each larger entry to its left



Initial

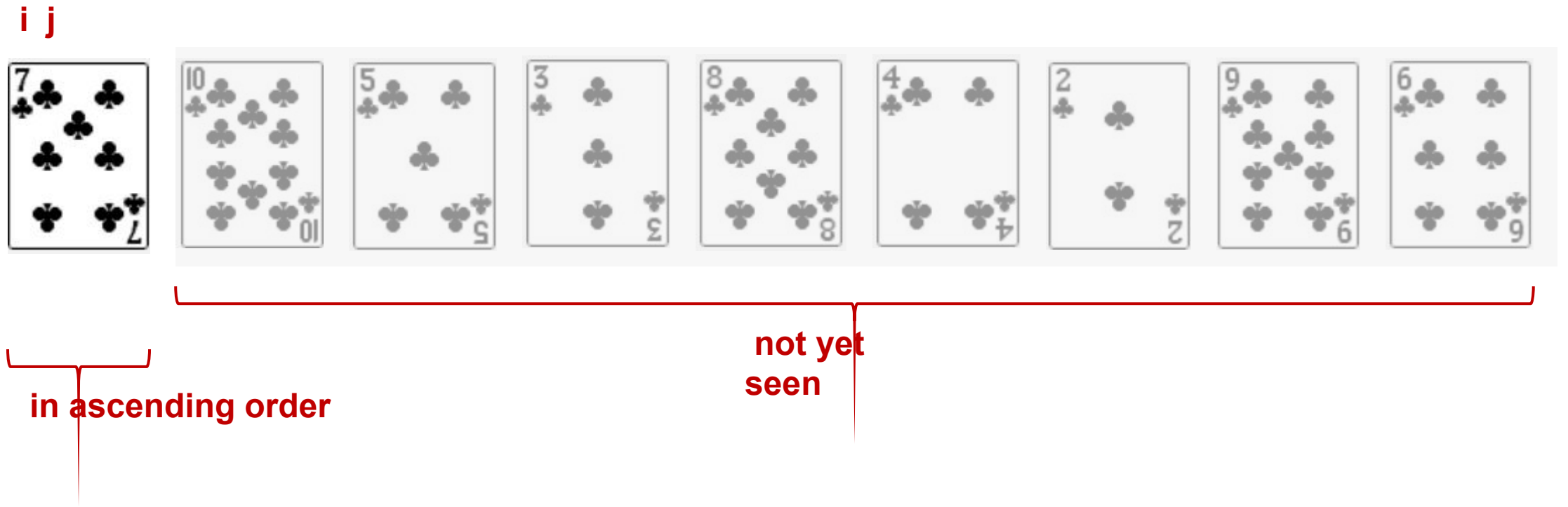
Insertion sort

- In iteration i , swap $a[i]$ with each larger entry to its left



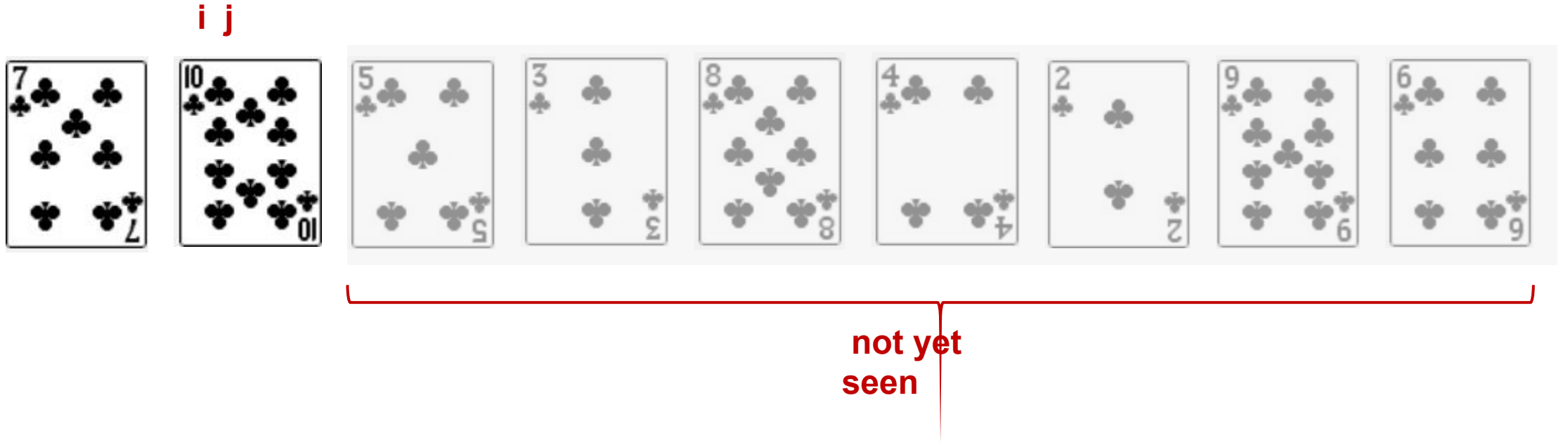
Insertion sort

- In iteration i , swap $a[i]$ with each larger entry to its left



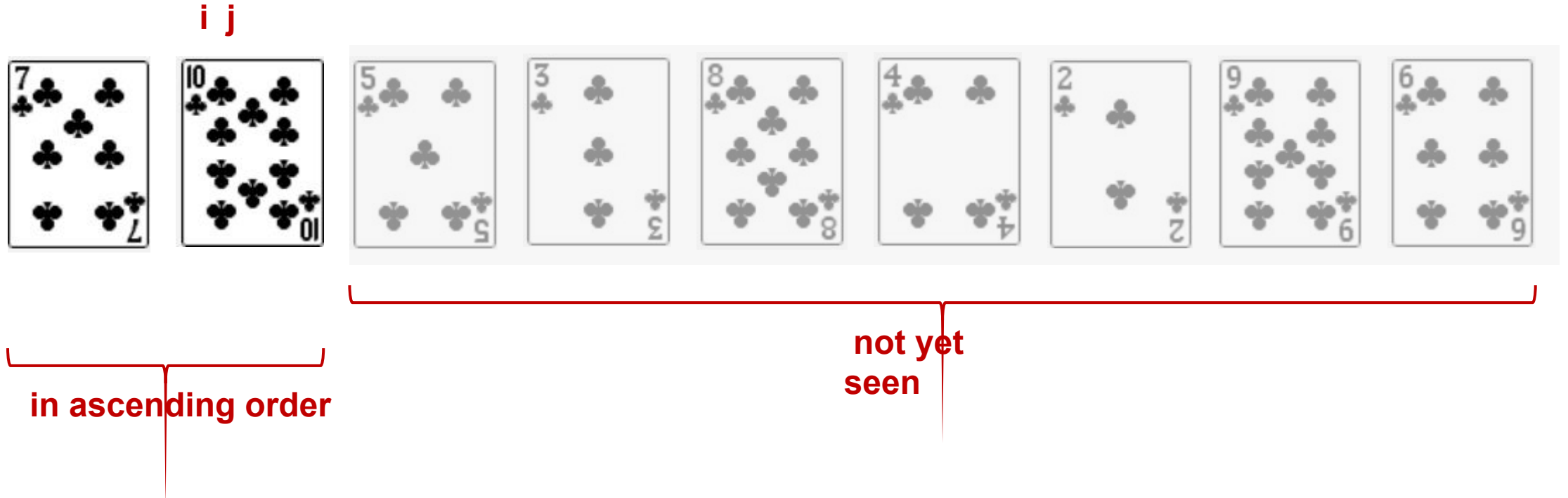
Insertion sort

- In iteration i , swap $a[i]$ with each larger entry to its left



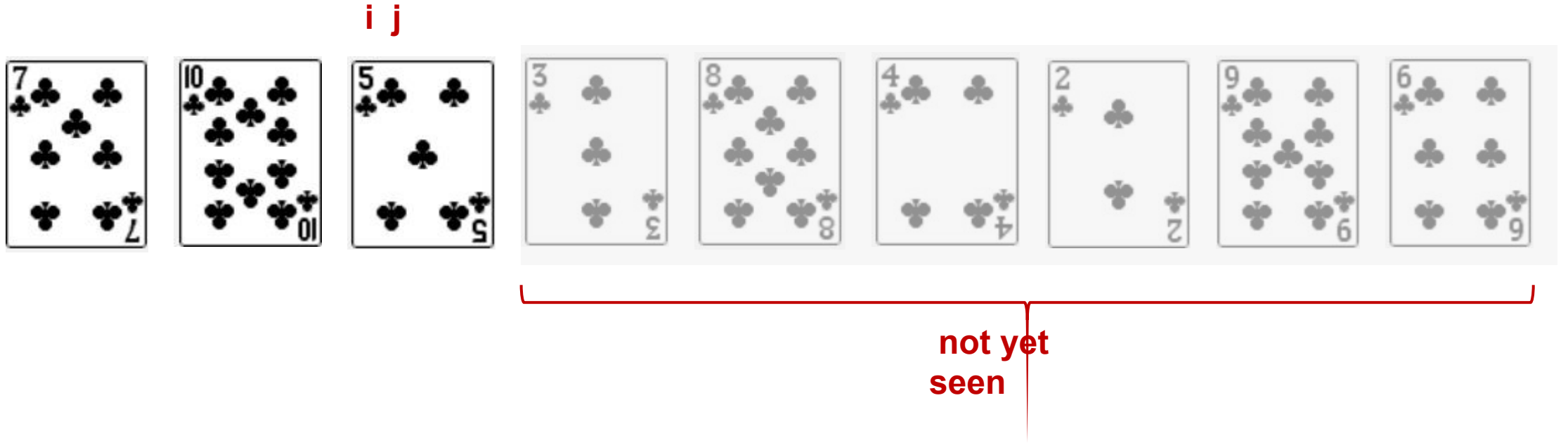
Insertion sort

- In iteration i , swap $a[i]$ with each larger entry to its left



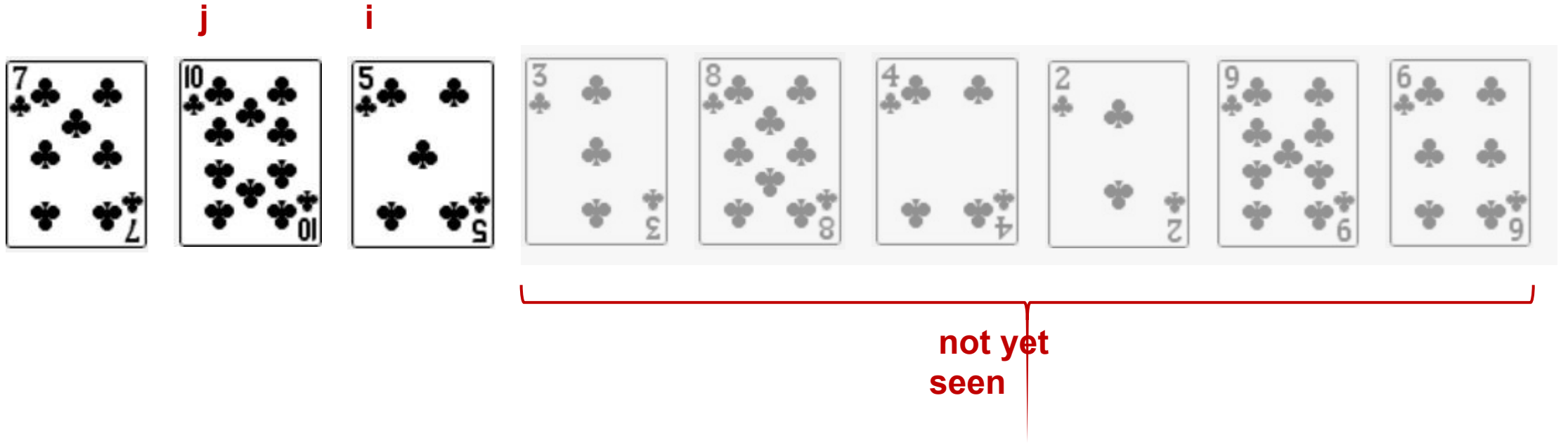
Insertion sort

- In iteration i , swap $a[i]$ with each larger entry to its left



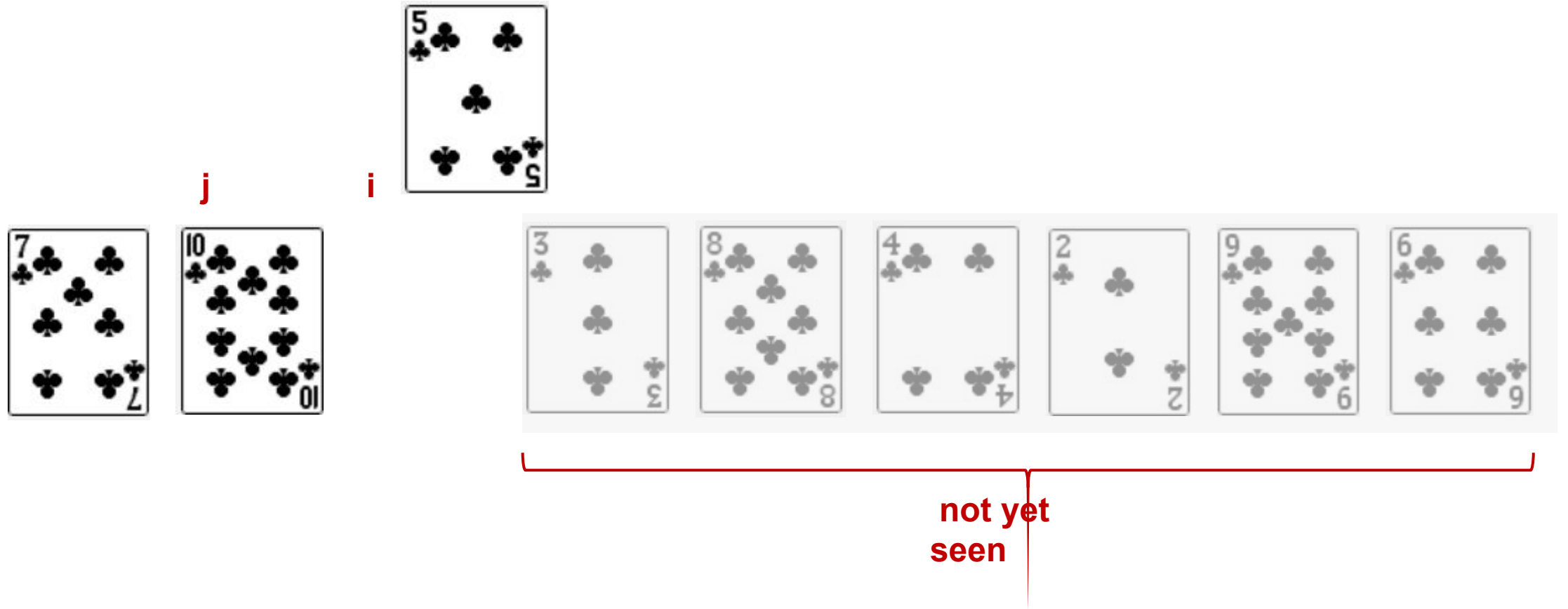
Insertion sort

- In iteration i , swap $a[i]$ with each larger entry to its left



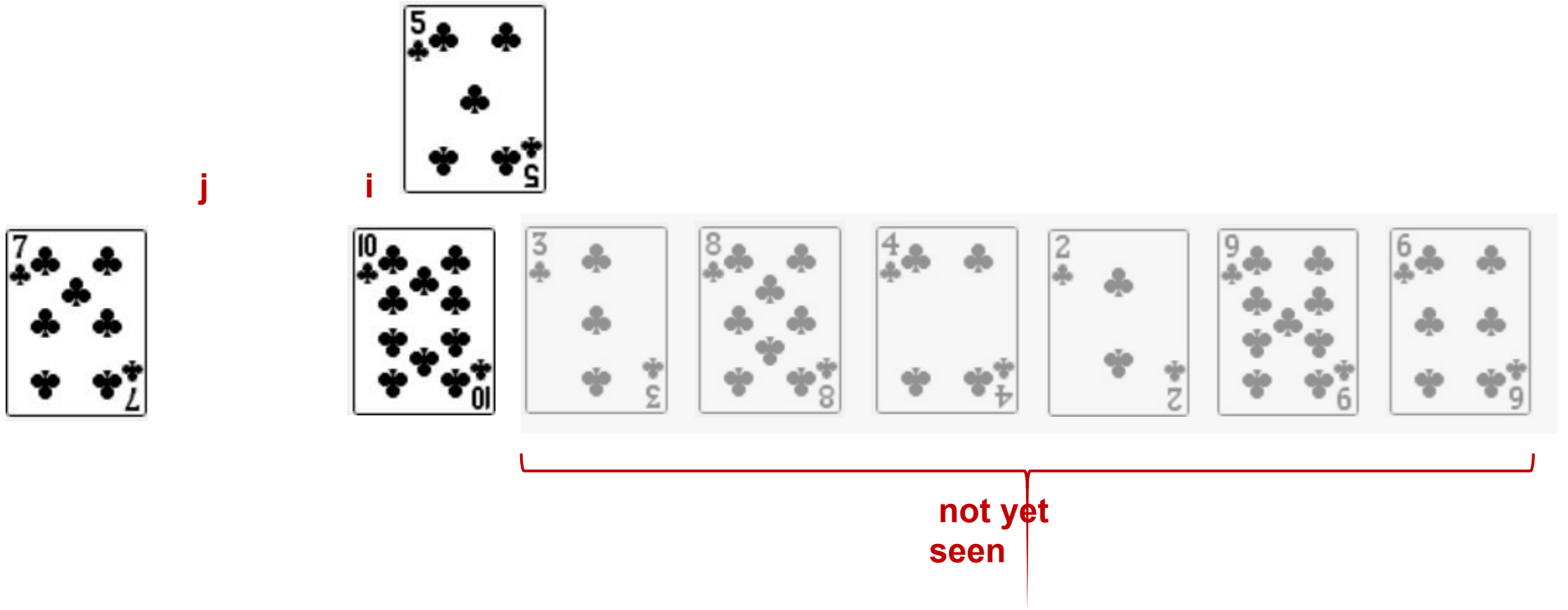
Insertion sort

- In iteration i , swap $a[i]$ with each larger entry to its left



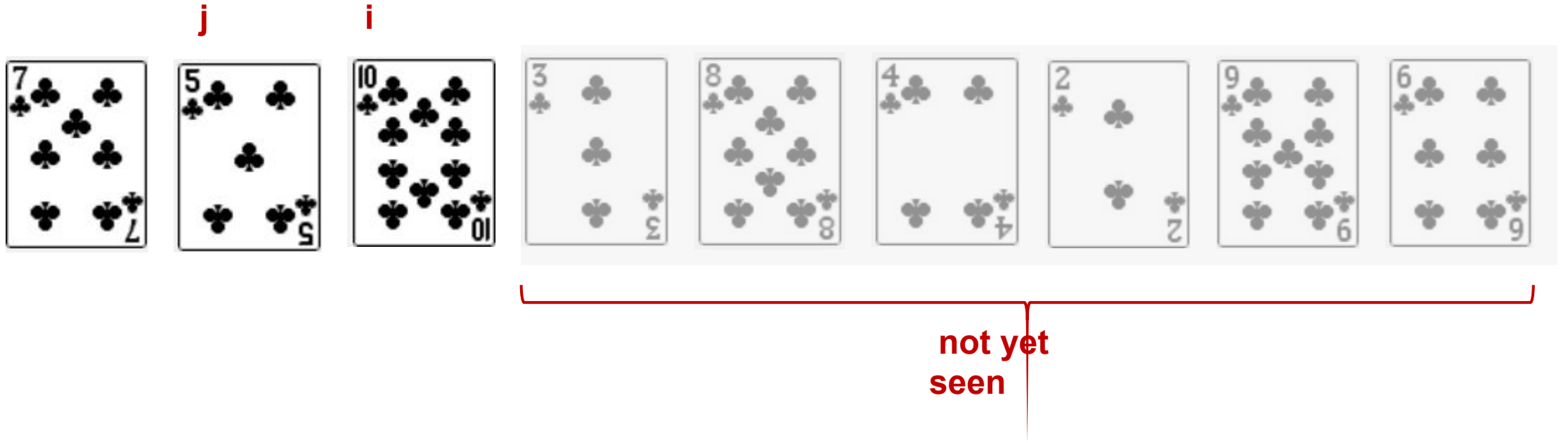
Insertion sort

- In iteration i , swap $a[i]$ with each larger entry to its left



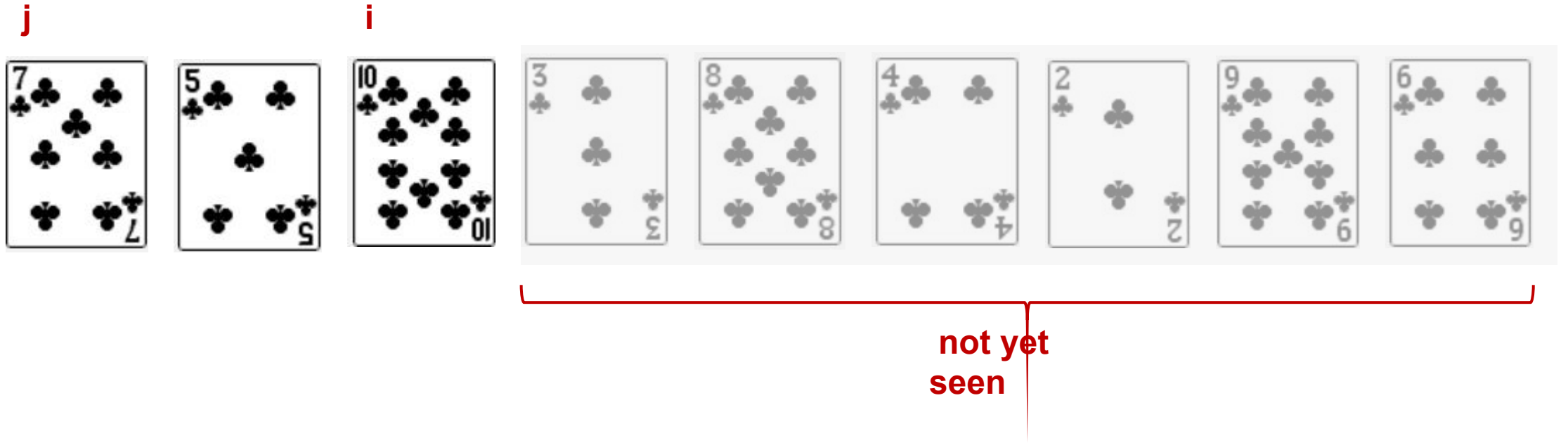
Insertion sort

- In iteration i , swap $a[i]$ with each larger entry to its left



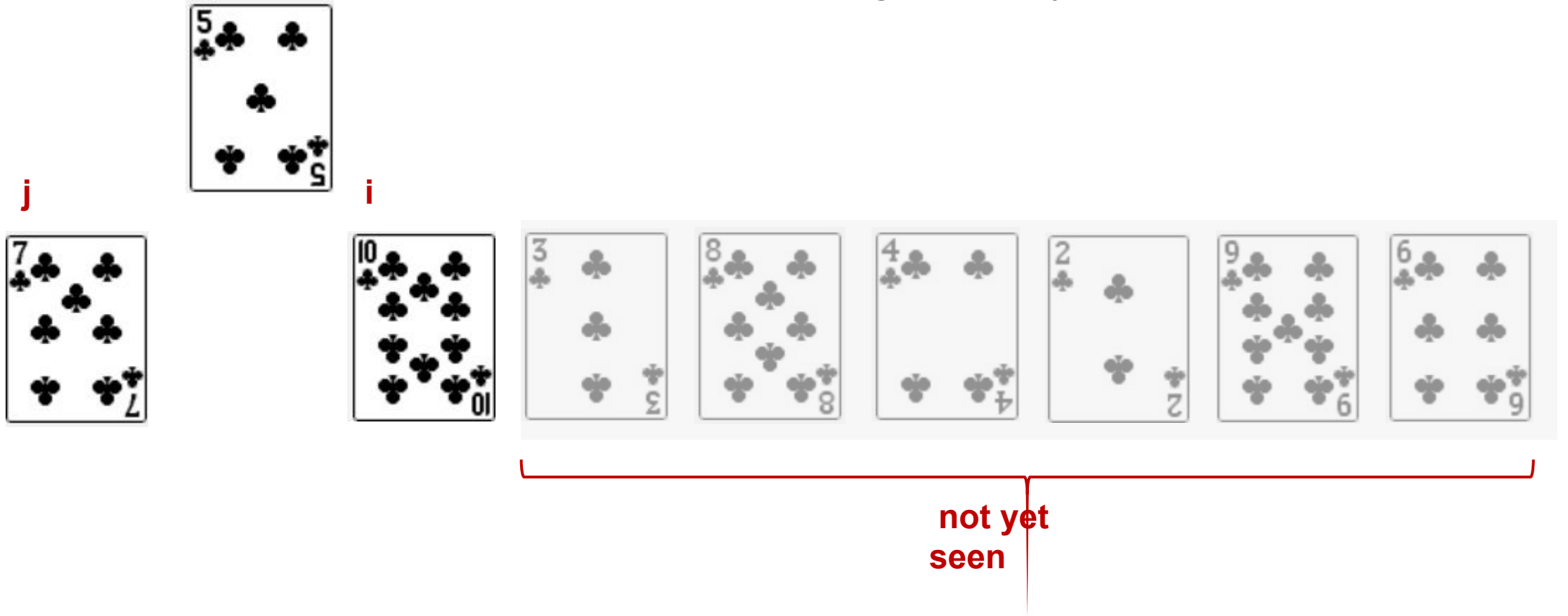
Insertion sort

- In iteration i , swap $a[i]$ with each larger entry to its left



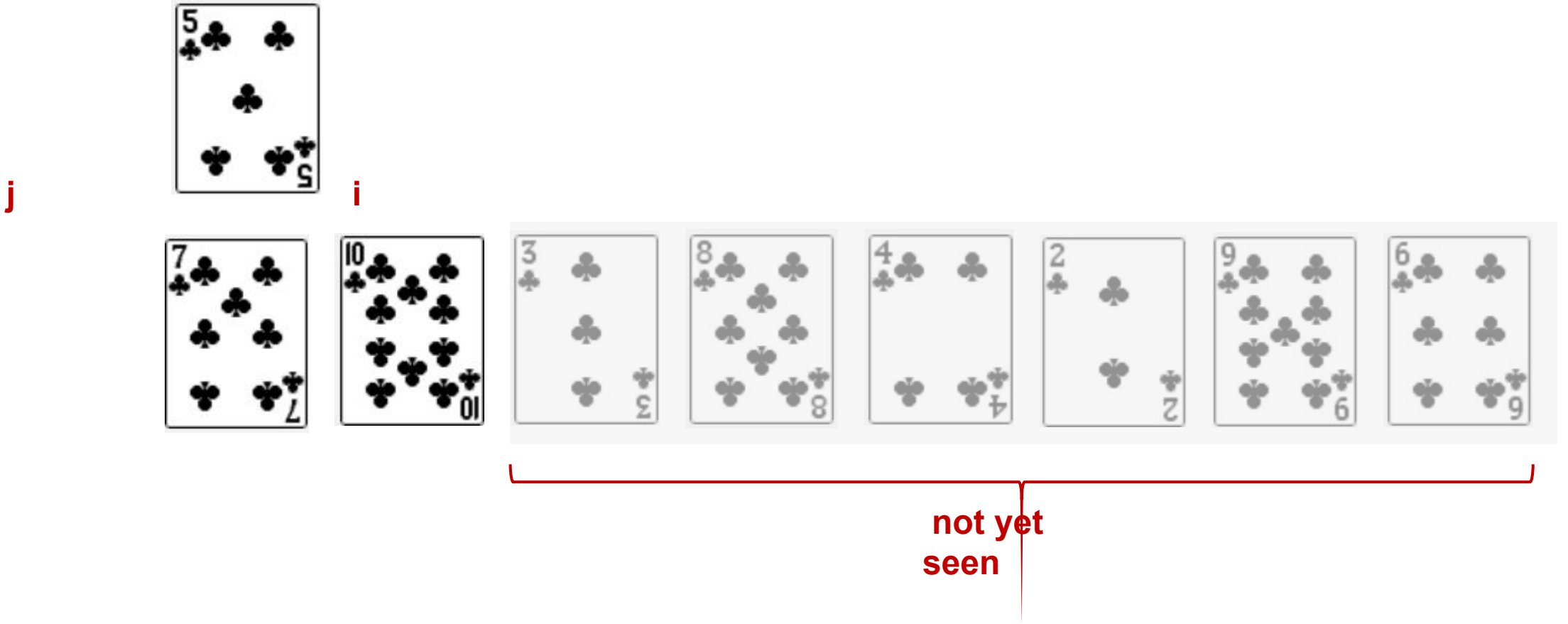
Insertion sort

- In iteration i , swap $a[i]$ with each larger entry to its left



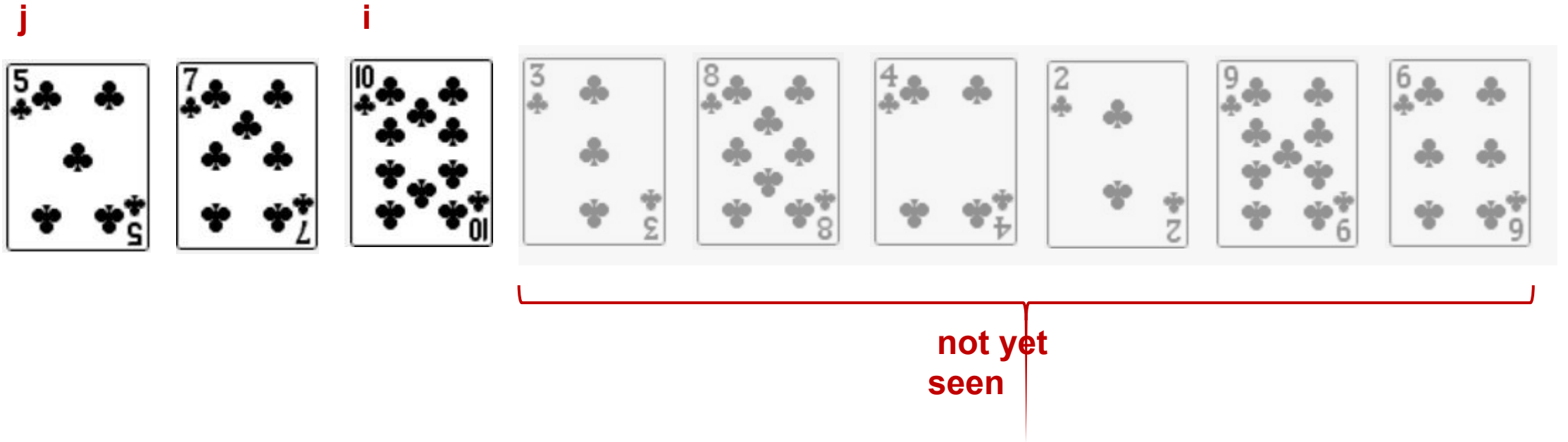
Insertion sort

- In iteration i , swap $a[i]$ with each larger entry to its left



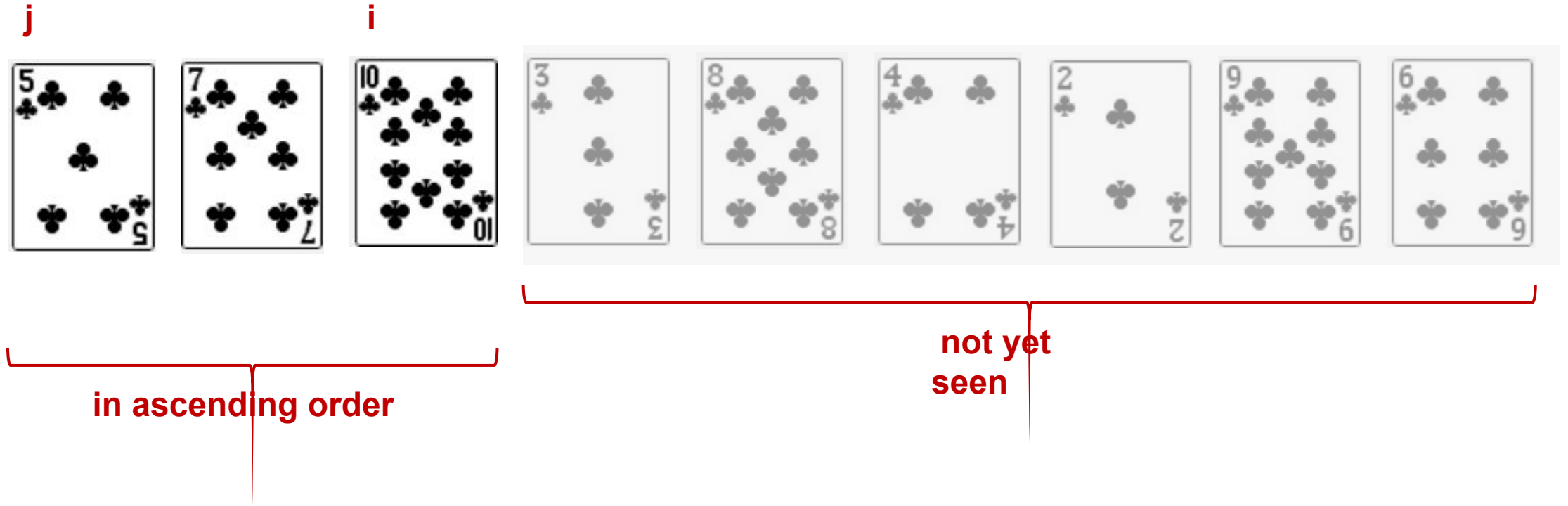
Insertion sort

- In iteration i , swap $a[i]$ with each larger entry to its left



Insertion sort

- In iteration i , swap $a[i]$ with each larger entry to its left

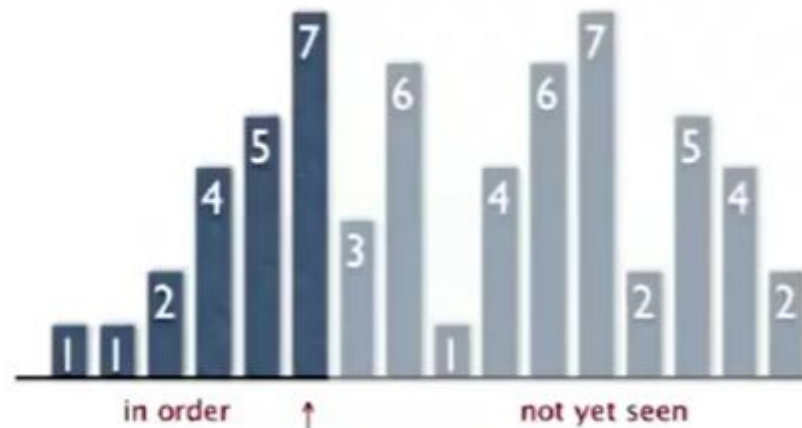


Insertion sort:

Algorithm: ↑ scan from left to right

Invariants:

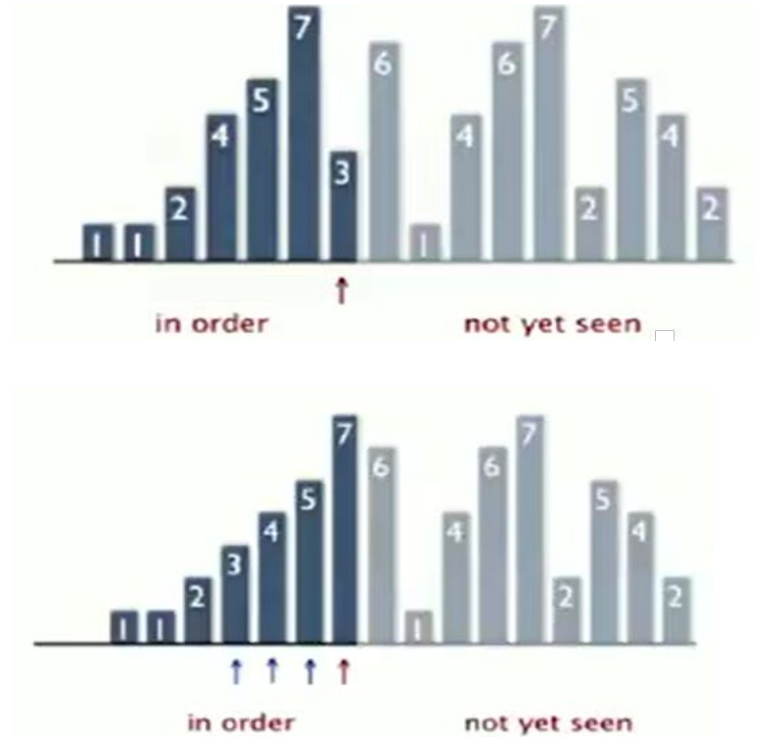
- Entries that left of ↑ (including ↑) are in ascending order.
- Entries to the right of ↑ have not yet been seen



Insertion sort inner loop

To maintain algorithm invariants:

- Move the pointer to the right
 $i += 1$
- Moving from right to left, exchange $a[i]$ with each larger entry to its left
for j in range($i, 0, -1$):
 if $a[j] < a[j-1]$
 $a[j], a[j-1] = a[j-1], a[j]$
 else: break



Insertion sort – Python implementation

```
def insertionSort(a):  
    n = a.size  
    for i in range(n):  
        for j in range(i, 0, -1):  
            if a[j] < a[j-1]:  
                a[j], a[j-1] = a[j-1], a[j]  
            else: break
```


Insertion sort: mathematical analysis

Proposition: To sort a random-ordered array with distinct keys, insertion sort uses $\sim \frac{1}{4}N^2$ compares and $\sim \frac{1}{4}N^2$ exchanges on average

Pf: Expect each entry to move halfway back

a[]

i	j		0	1	2	3	4	5	6	7	8	9	10
			S	O	R	T	E	X	A	M	P	L	E
1	0		O	S	R	T	E	X	A	M	P	L	E
2	1		O	R	S	T	E	X	A	M	P	L	E
3	3		O	R	S	T	E	X	A	M	P	L	E
4	0		E	O	R	S	T	X	A	M	P	L	E
5	5		E	O	R	S	T	X	A	M	P	L	E
6	0		A	E	O	R	S	T	X	M	P	L	E
7	2		A	E	M	O	R	S	T	X	P	L	E
8	4		A	E	M	O	P	R	S	T	X	L	E
9	2		A	E	L	M	O	P	R	S	T	X	E
10	2		A	E	E	L	M	O	P	R	S	T	X
			A	E	E	L	M	O	P	R	S	T	X

entries in gray do not move

entries in red is a[j]

entries in black moved one position right for insertion

Trace of insertion sort (array content just after each insertion)

i	j	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34		
		A	S	O	M	E	W	H	A	T	L	O	N	G	E	R	I	N	S	E	R	T	I	O	N	S	O	R	T	E	X	A	M	P	L	E		
0	0	A																																				
1	1		S																																			
2	1		O	S																																		
3	1		M	O	S																																	
4	1		E	M	O	S																																
5	5					W																																
6	2			H	M	O	S	W																														
7	1		A	E	H	M	O	S	W																													
8	7								T	W																												
9	4				L	M	O	S	T	W																												
10	7								O	S	T	W																										
11	6								N	O	O	S	T	W																								
12	3				C	H	L	M	N	O	O	S	T	W																								
13	3				E	C	H	L	M	N	O	O	S	T	W																							
14	11												R	S	T	W																						
15	6									I	L	M	N	O	O	R	S	T	W																			
16	10												N	O	O	R	S	T	W																			
17	15																	S	T	W																		
18	4				E	C	H	I	L	M	N	N	O	O	R	S	S	S	T	W																		
19	15																		R	S	S	T	W															
20	19																			I	W																	
21	8										I	L	M	N	N	O	O	R	R	S	S	S	T	T	W													
22	15																		O	R	R	S	S	S	T	T	W											
23	13													N	O	O	R	R	S	S	S	T	T	W														
24	21																				S	T	T	W														
25	17																		O	R	R	S	S	S	S	T	T	W										
26	20																					R	S	S	S	S	T	T	W									
27	26																													I	W							
28	5				E	C	H	I	I	L	M	N	N	N	O	O	O	O	R	R	R	S	S	S	S	T	T	T	T	W								
29	29																																	X				
30	2		A	E	E	E	E	C	H	I	I	L	M	N	N	N	O	O	O	O	R	R	R	S	S	S	S	T	T	T	T	W	X					
31	13														M	N	N	N	O	O	O	O	R	R	R	S	S	S	S	T	T	T	T	W	X			
32	21																					P	R	R	R	S	S	S	S	T	T	T	T	W	X			
33	12														L	M	M	N	N	N	O	O	O	O	P	R	R	R	S	S	S	S	T	T	T	W	X	
34	7						E	C	H	I	I	L	L	M	M	N	N	N	N	O	O	O	O	P	R	R	R	S	S	S	S	T	T	T	W	X		
		A	A	A	E	E	E	E	E	C	H	I	I	L	L	M	M	N	N	N	N	O	O	O	O	P	R	R	R	S	S	S	S	T	T	T	W	X

Few demos from google seach

Insertion sort: best and worst case

- Best case: if the array is in ascending order, insertion sort makes $N-1$ compare and 0 exchange

A E E L M O P R S T X

- Worst case: If the array is in descending order (and no duplicates), insertion sort makes $\sim \frac{1}{2} N^2$ compare and $\sim \frac{1}{2} N^2$ exchanges

Z T S R P O M L E E A

QUIZ

How many compares does insertion sort make on an input array that is already sorted?

- a) Constant
- b) Logarithmic
- c) Linear
- d) quadratic

QUIZ

How many compares does insertion sort make on an input array that is already sorted?

- a) Constant
- b) Logarithmic
- c) Linear
- d) Quadratic

Each item (except the first) is compared against the item to its left (and to no other items), for a total of $n-1$ compares. And: linear

Preview for Shellsort

Idea: Move entries more than one position at a time by h-sorting the array

an h-sorting array is h interleaved sorting subsequence

$h = 4$

L E E A M H L E P S O L T S X R

L		M		P		T			
	E		H		S		S		
		E		L		O		X	
			A		E		L		R

Preview Shellsort

Shellsort (Shell: 1956) h-sort array for decrease sequence of values of h

0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5

input - S H E L L S O R T E X A M P L E

13-sort P H E L L S O R T E X A M S L E

4-sort L E E A M H L E P S O L T S X R

1-sort A E E E H L L L M O P R S S T X